Meadow9 iOS & Mobile App Development

Interim Report

Toh Kar Le
U2221258J

Supervisor: Dr Chng Eng Siong

College of Computing and Data Science
Nanyang Technological University

Academic Year 2025/2026

Abstract

Long-duration meeting transcription systems have become increasingly prevalent in both enterprise and professional settings. While speech-to-text accuracy has improved substantially, effective and efficient retrieval of information from lengthy transcripts remains a significant challenge. Meadow9+, a long-meeting transcription application developed by ABAX AI, recently introduced a chatbot feature to address this need; however, its current implementation processes entire transcripts directly as input to a large language model (LLM), resulting in high latency, excessive computational cost, and degraded answer quality due to long-context limitations.

This project proposes and develops a Retrieval-Augmented Generation (RAG)–based system to optimize transcript retrieval and chatbot performance for Meadow9+. By embedding transcript segments into a vector space and retrieving only the most relevant contextual chunks for each user query, the proposed system significantly reduces inference overhead while preserving answer accuracy. In addition, this work introduces a comprehensive evaluation pipeline that systematically assesses both retrieval quality and end-to-end question-answering performance using metrics such as Hit Rate, Mean Reciprocal Rank (MRR), F1 score, and overall accuracy, alongside detailed latency benchmarking.

Experimental results demonstrate that the RAG-based chatbot achieves comparable accuracy to the existing full-transcript LLM approach while delivering substantial improvements in response time, validating the effectiveness of retrieval-driven optimization. Beyond immediate chatbot acceleration, the embedding-based architecture enables extensibility toward more advanced retrieval techniques, including phonetic search, fuzzy matching, and hybrid retrieval strategies tailored to noisy speech-to-text outputs. This report presents the system design, implementation progress, and evaluation results to date, and outlines future work focused on building a comprehensive, efficient, and deployable retrieval system.

Contents

Abstract **1**

1 Introduction **7**
1.1 Background . . . 7
1.2 Motivations and Objectives . . . 7
1.3 Scope . . . 8
1.4 Contributions . . . 9
1.5 Project Schedule . . . 10

2 Literature Review **12**
2.1 Otter.ai . . . 12
2.1.1 Retrieval Strategies . . . 12
2.1.2 Accuracy and Latency Characteristics . . . 14
2.1.3 Transparency of Retrieval and Reasoning . . . 14
2.1.4 Discussion and Implications . . . 14
2.2 Claude Code . . . 15
2.2.1 Retrieval Strategies . . . 15
2.2.2 Accuracy and Latency Characteristics . . . 16
2.2.3 Transparency of Retrieval and Reasoning . . . 16
2.2.4 Discussion and Implications . . . 17
2.3 Pre-Project Meadow9+ . . . 18
2.3.1 Retrieval Strategies . . . 18
2.3.2 Accuracy and Latency Characteristics . . . 19
2.3.3 Transparency of Retrieval and Reasoning . . . 19
2.3.4 Discussion and Implications . . . 20

3 Proposed System Architecture **21**
3.1 System Overview . . . 21
3.2 Indexing Phase . . . 21
3.3 Querying Phase . . . 22
3.4 Hybrid Retrieval System . . . 23
3.5 Evaluation Architecture . . . 23

4 Implementation and Progress to Date **26**
4.1 External Resources Incorporated . . . 26
4.1.1 Core Framework Libraries . . . 26
4.1.2 Models Integration . . . 26
4.1.3 Other Libraries . . . 26

4.2 Summarization Pipeline Implementation 27
4.2.1 Transcript Chunking for Summarization 27
4.2.2 Per-Chunk Structured Extraction 28
4.2.3 Hierarchical Merge and Canonical Summary Schema 28
4.2.4 Summary Embedding Documents 28
4.3 RAG Pipeline Implementation 29
4.3.1 Transcript Chunking Implementation 29
4.3.2 Embedding Generation and Vector Storage 30
4.3.3 Multi-Stage Retrieval Process 33
4.3.4 Context Assembly and Prompt Construction 40
4.4 Keyword-Based Search Implementation 44
4.4.1 Keyword-First Retrieval Design 44
4.4.2 Two-Phase Search Flow 44
4.4.3 Date and Speaker Filtering 45
4.4.4 Relationship to the RAG Subsystem 45
4.5 Stretch Goals and Deployment 45

5 Evaluation and Results **46**
5.1 Evaluation Pipeline Implementation 46
5.1.1 Dataset Generation Methodology 46
5.1.2 Retrieval Metrics 49
5.1.3 Generation Metrics 51
5.1.4 Separation of Retrieval and Generation Evaluation 53
5.1.5 Speed Evaluation Pipeline 55
5.1.6 Statistical Computation 55
5.2 Experimental Results 56
5.2.1 RAG Quality Metrics 56
5.2.2 Speed Performance 61

6 Conclusion and Future Work **63**
6.1 Future Work 63
6.1.1 Limited Coverage of Question Types in the Evaluation Pipeline . 63
6.1.2 RAG Pipeline Optimization for Ambiguous Queries 64
6.1.3 Production Deployment of Retrieval Server 64

Bibliography **68**

List of Figures

1.1 Gantt chart representing the project timeline 10

2.1 Otter.ai iOS application interface showing keyword search, AI chat, and summarization features 13

3.1 System integration: transcript forwarding, summary return, and retrieval indexing 22
3.2 Indexing phase: transcript processing, chunking, and embedding stages . 22
3.3 Querying phase: retrieval and response generation 23
3.4 Querying phase: high-level query-to-response architecture 23
3.5 Evaluation pipeline for the RAG question-answering subsystem 24
3.6 Benchmarking: RAG-based QA subsystem vs. pure LLM baseline 25

4.1 Placeholder for the implemented summarization pipeline 28
4.2 Placeholder for summary embedding document generation 29
4.3 SRT input format with sequence numbers and timestamps 29
4.4 Parsed SRT structure with speaker metadata extraction 30
4.5 Token-based merging algorithm for chunk size optimization 30
4.6 Document processing pipeline 31
4.7 Vector store creation from documents 32
4.8 Incremental document addition 32
4.9 ChromaDB storage directory structure 32
4.10 Vector search core function showing semantic retrieval with optional metadata filtering 33
4.11 BM25 tokenization function using lowercase split 33
4.12 BM25 index initialization with corpus tokenization 34
4.13 BM25 query tokenization and metadata filtering 34
4.14 Phonetic word tokenization and filtering 35
4.15 Double Metaphone phonetic encoding 35
4.16 Phonetic encoding examples for homophone matching 35
4.17 Phonetic retrieval SQL query with transcript filtering 36
4.18 Phonetic score conversion and document retrieval 36
4.19 Rare token detection using Zipf frequency 37
4.20 Extracting rare tokens from query 37
4.21 Fuzzy scoring algorithm: setup and initialization 37
4.22 Fuzzy scoring algorithm: nested loop with RapidFuzz ratio 38
4.23 Fuzzy top-k selection and ranking 38
4.24 Reciprocal Rank Fusion weights for the hybrid retrieval signals 39
4.25 Expected reranker response format 40

4.26 Mapping reranked results back to the original documents 40
4.27 Top-n selection after reranking 40
4.28 Chunk formatting with individual metadata headers 41
4.29 Output format example 41
4.30 Citation formatting for user transparency and source attribution 42
4.31 Grounded QA system message template defining assistant role and constraints 43
4.32 Complete RAG chat prompt construction with history support 43
4.33 Chat history structure for multi-turn conversation support 43
4.34 Streaming response handling for real-time token display 44
4.35 Placeholder for the implemented keyword-based search flow 45

5.1 Example transformation from raw SRT transcript chunks to a grouped window with temporal metadata 47
5.2 LangChain pipe syntax for QA generation chain 48
5.3 QA generation prompt template with requirements and format 48
5.4 Ground truth JSON format example with timestamps 49
5.5 Hit Rate formula for retrieval evaluation 50
5.6 Interval intersection algorithm for retrieval relevance checking 50
5.7 Mean Reciprocal Rank (MRR) formula and definition 51
5.8 F1 score calculation with precision and recall 51
5.9 Text preprocessing for token-level comparison 52
5.10 Precision, recall, and harmonic mean calculation with Counter intersection 52
5.11 LLM judge prompt template for semantic correctness evaluation 53
5.12 Retrieval failure scenario diagnostic example 54
5.13 Generation failure scenario with successful retrieval 54
5.14 Perfect performance scenario for both retrieval and generation 54
5.15 Interleaved execution pattern for unbiased speed benchmarking 55
5.16 Detailed success case example with perfect retrieval and generation . . . 58
5.17 Failure case example with retrieval success but generation error 59
5.18 False negative example with more specific generated answer 60

List of Tables

1.1 Detailed project schedule 11

4.1 Third-party libraries and frameworks used across the hybrid retrieval system 27
4.2 Search method characteristics in the hybrid retrieval stage used by the RAG subsystem 39

5.1 Ground truth schema field descriptions 49
5.2 Statistical metrics table for latency analysis 56
5.3 Comparative metrics between RAG and Pure LLM approaches 56
5.4 Per-transcript RAG quality metrics aggregated results 57
5.5 Per-transcript speed comparison showing speedup factors 61
5.6 Quality-speed trade-off analysis across transcript sizes 61

Chapter 1

Introduction

1.1 Background

Large Language Models (LLMs) represent a class of deep learning architectures trained on massive datasets, enabling them to comprehend and generate natural language with high versatility across domains [1]. As of 2025, approximately 67% of organizations worldwide have adopted LLMs to support their operational workflows [2]. One of the most prominent applications of LLMs is in conversational agents or chatbots [3]. These systems often integrate retrieval pipelines [4] that extract domain-specific information from a database or knowledge store, and provide the retrieved content as context to the LLM for response generation. This architecture allows users to query against proprietary or organizational datasets using natural language and receive accurate, context-specific responses.

Meadow9+, a speech recognition application designed to record and transcribe long-duration meetings, is one such platform that integrates an LLM-powered chatbot to enable retrieval of information from meeting transcripts. The platform is widely utilised by corporations, welfare organizations, and medical professionals who rely on accurate transcription of meetings. Since meetings are where critical discussions and decisions take place, these transcripts effectively serve as a knowledge base for the organization. The integrated chatbot therefore plays a crucial role in extracting actionable insights from this knowledge, and the system must ensure both accuracy and responsiveness of the retrieval process.

Currently, the Meadow9+ chatbot processes full transcripts directly as input to the LLM. This design leads to two major limitations. First, excessively long inputs exacerbate the "lost in the middle" phenomenon [5], where the LLM exhibits degraded performance when reasoning over information embedded in the middle sections of extended context windows. Second, longer context sequences increase computational overhead, thereby slowing inference and degrading user experience [6]. Furthermore, transcription outputs often contain inaccuracies and inconsistencies [7], and the current chatbot system lacks functionalities to handle noisy and ambiguous transcript data. These factors reduce usability and limit the effectiveness of the retrieval process for users who need quick and accurate access to meeting information.

1.2 Motivations and Objectives

Meeting transcripts only become truly valuable and actionable when users can quickly locate decisions, tasks, and explanations. However, Meadow9+ currently lacks an effective

retrieval mechanism to support such use cases, which significantly limits the practical utility of long-duration transcripts. As mentioned, the existing Meadow9+ chatbot employs a monolithic prompting approach, in which entire transcripts are provided directly to the LLM. This design leads to high latency, increased computational overhead, and degraded user experience, rendering the system unreliable in real-world scenarios. Improving information retrieval within Meadow9+ therefore directly addresses a critical gap in the current system and has immediate applicability in real-world deployments.

This project presents an opportunity to apply modern information retrieval (IR) and LLM integration techniques beyond naive prompting. In particular, Retrieval-Augmented Generation (RAG) combines classical retrieval methods with generative models, enabling systems to retrieve relevant document segments before generation [8]. Furthermore, the project also enables exploration of advanced retrieval enhancements, including query preprocessing, phonetic encoding, and fuzzy matching.

Motivated by these considerations, this project aims to design and implement a comprehensive transcript retrieval system that improves both retrieval accuracy and end-to-end system performance for long meeting transcripts. Specifically, the objectives are:

- Build a transcript retrieval pipeline that indexes long meeting transcripts into retrievable units and supports semantic retrieval via embeddings.
- Integrate retrieval into the chatbot using a RAG workflow to avoid full-transcript prompting, improving latency while maintaining answer quality.
- Develop a comprehensive evaluation pipeline to measure retrieval quality and end-to-end question answering performance using standard metrics, including hit-based metrics, reciprocal-rank metrics, and QA-style correctness metrics.
- Conduct empirical benchmarking comparing RAG against the existing pure LLM baseline under realistic conditions, with a focus on both accuracy and response latency.
- Explore extensible retrieval techniques tailored to noisy ASR transcripts, enabling efficient date-based and keyword-based retrieval in addition to chatbot-driven querying.

1.3 Scope

The scope of this project primarily focuses on the design and implementation of a RAG pipeline tailored for long-duration meeting transcripts. This includes developing appropriate transcript preprocessing strategies, such as chunking and embedding, to enable efficient and scalable retrieval from large textual documents. Particular emphasis is placed on handling the characteristics of long-form transcripts, where individual segments may vary in length and informational density, and ensuring that the resulting representations are suitable for downstream retrieval and generation tasks.

In addition to system design, the project places strong emphasis on quantitative evaluation and performance analysis. Retrieval effectiveness is systematically assessed using established information retrieval metrics, including Hit Rate, Mean Reciprocal Rank (MRR) [9], F1-score, and Accuracy, to provide an objective measure of retrieval quality. Furthermore, speed benchmarking is conducted to evaluate system efficiency, covering both end-to-end response latency and component-level performance across the retrieval and generation pipeline. The scope also includes exploratory work on system extensibility, laying the groundwork for integrating advanced retrieval mechanisms that can further enhance robustness and recall in noisy transcript environments.

Several aspects are explicitly considered out of scope to maintain a clear focus on retrieval and chatbot optimization. Improvements to the underlying Automatic Speech Recognition (ASR) model are excluded, as the project assumes transcripts are provided by the existing Meadow9+ transcription system. Similarly, user interface and user experience (UI/UX) redesign of the Meadow9+ application is not addressed. Finally, the project does not involve training or fine-tuning custom large language models, and instead focuses on optimizing system-level integration and retrieval strategies using pre-existing LLMs.

1.4 Contributions

Note that this section only provides an overview of the key contributions made. Additional details will be discussed in subsequent chapters, where they are grouped and presented differently. A summary of key contributions to this project is presented below:

Developed a hybrid transcript retrieval pipeline that indexes long-form meeting transcripts using token-based chunking and combines semantic, lexical search, phonetic search, and fuzzy matching. The pipeline employs Reciprocal Rank Fusion (RRF) [10] and cross-encoder reranking [11] to surface the top-5 most relevant chunks, achieving 96.7% Hit Rate and 0.941 Mean Reciprocal Rank across six diverse transcripts, demonstrating highly accurate retrieval without manual segmentation.

Integrated the retrieval pipeline into a RAG-based chatbot using LangChain orchestration, replacing full-transcript prompting with context-aware retrieval. This approach leverages per-transcript metadata filtering and temporal alignment, delivering an average 5.39× speedup over the pure LLM baseline, directly addressing latency and token cost constraints in production conversational interfaces.

Implemented a comprehensive evaluation framework measuring both retrieval quality (Hit Rate, MRR) and generation quality (F1 score, LLM-as-judge accuracy). The framework validates temporal overlap between retrieved chunks and ground-truth spans, providing multi-dimensional assessment of RAG system performance. Evaluation on 62 automatically generated QA pairs demonstrates 78.9% semantic correctness while maintaining sub-second retrieval latency (6% of total response time).

Conducted controlled empirical benchmarking comparing RAG against a pure LLM baseline under realistic conditions. Component-level timing measurements isolate retrieval overhead (mean: 0.42–0.55s) from generation time, and interleaved query ex-

ecution mitigates prompt caching bias. Results demonstrate that RAG achieves 78.9% accuracy at 5.39× speedup compared to baseline, establishing practical viability for real-time transcript question-answering in resource-constrained environments.

Implemented a global keyword-first search bar as an extensible retrieval mechanism tailored to noisy ASR transcripts, enabling efficient date-based, speaker-based, and keyword-based retrieval in addition to chatbot-driven querying. The system employs a two-phase pipeline that ranks transcripts globally before running per-transcript chunk retrieval with cross-encoder reranking, with phonetic and fuzzy matching to handle ASR transcription noise.

1.5 Project Schedule

The detailed project schedule is presented in Table 1.1, with the corresponding timeline visualization shown in Figure 1.1.

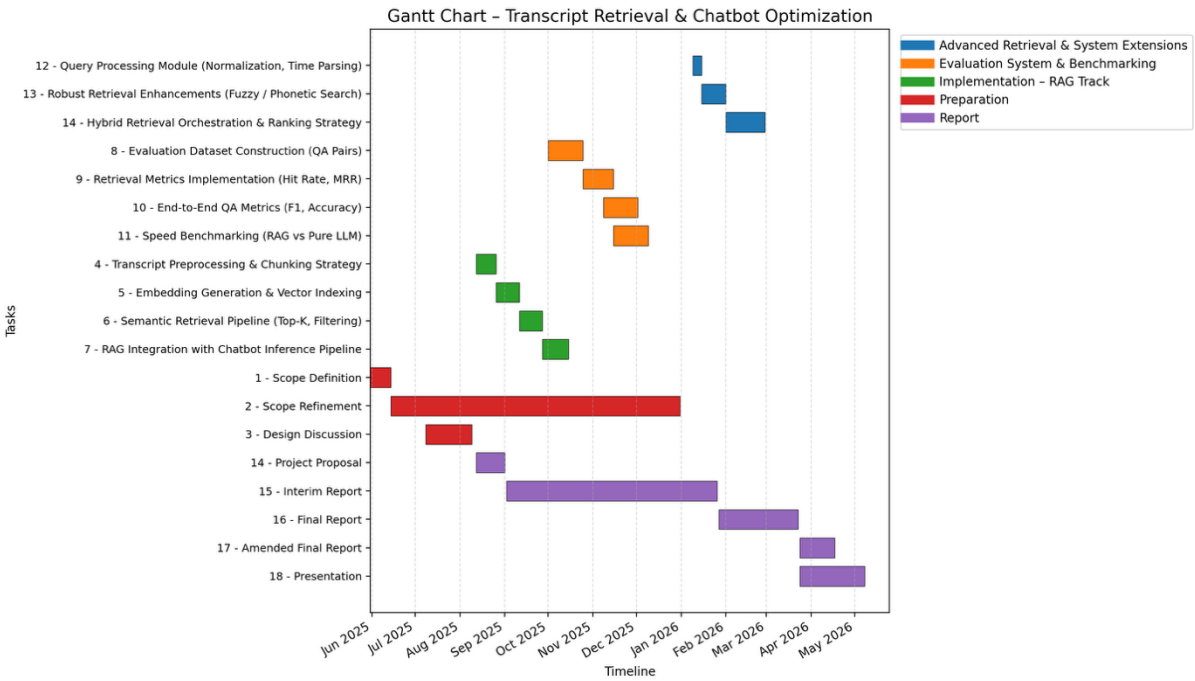

Figure 1.1: Gantt chart representing the project timeline

Index	Task Name	Start Date	End Date
Preparation			
1	Scope Definition	31 May 2025	14 Jun 2025
2	Scope Refinement	14 Jun 2025	31 Dec 2025
3	Design Discussion	8 Jul 2025	9 Aug 2025
Implementation – RAG Track			
4	Transcript Preprocessing & Chunking Strategy	12 Aug 2025	26 Aug 2025
5	Embedding Generation & Vector Indexing	26 Aug 2025	11 Sept 2025
6	Semantic Retrieval Pipeline (Top-K, Filtering)	11 Sept 2025	27 Sept 2025
7	RAG Integration with Chatbot Inference Pipeline	27 Sept 2025	15 Oct 2025
Evaluation System & Benchmarking			
8	Evaluation Dataset Construction (QA Pairs)	1 Oct 2025	25 Oct 2025
9	Retrieval Metrics Implementation (Hit Rate, MRR)	25 Oct 2025	15 Nov 2025
10	End-to-End QA Metrics (F1, Accuracy)	8 Nov 2025	2 Dec 2025
11	Speed Benchmarking (RAG vs Pure LLM)	15 Nov 2025	9 Dec 2025
Advanced Retrieval & System Extensions			
12	Query Processing Module (Normalization, Time Parsing)	9 Jan 2026	15 Jan 2026
13	Robust Retrieval Enhancements (Fuzzy / Phonetic Search)	15 Jan 2026	1 Feb 2026
14	Hybrid Retrieval Orchestration & Ranking Strategy	1 Feb 2026	28 Feb 2026
Report			
14	Project Proposal	12 Aug 2025	1 Sept 2025
15	Interim Report	2 Sept 2025	26 Jan 2026
16	Final Report	27 Jan 2026	23 Mar 2026
17	Amended Final Report	24 Mar 2026	17 Apr 2026
18	Presentation	24 Mar 2026	8 May 2026

Table 1.1: Detailed project schedule

Chapter 2

Literature Review

In today's competitive landscape, modern transcription-based applications and meeting asssistants increasingly integrate chat-based interfaces to allow users to query recorded conversations. In approaching the development of this project, a comprehensive review was conducted on the retrieval capabilities of three existing products: Otter.ai, Claude Code and the existing Meadow9+ mobile application. These products were selected due to their relevance to the proposed project.

The literature review focused on three key aspects: retrieval strategies, accuracy and latency characteristics, and transparency of retrieval and reasoning. Instead of trying to replicate existing solutions, this review aimed to understand how these platforms effectively address retrieval challenges and identify opportunities for improvement in developing our project. An in-depth analysis of the technological stacks and architectures will be presented in later chapters, specifically in the context of Meadow9+, as such information is not publicly available for the other products reviewed.

This chapter is divided into three main sections. Section 2.1 reviews Otter.ai, and Section 2.2 examines Claude Code, focusing exclusively on their retrieval system. Section 2.3 evaluates the Meadow9+ mobile application prior to this project.

2.1 Otter.ai

Otter.ai is a widely adopted speech-to-text and meeting assistant platform that provides real-time transcription, post-meeting summaries, and multiple mechanisms for retrieving information from recorded conversations [12]. As a mature commercial system, it represents a strong baseline for evaluating existing approaches to transcript retrieval and conversational interaction.

Otter.ai caters to a wide range of users, including businesses and students [13, 14]. While it is primarily used for transcribing meetings, the platform also proves valuable for sales calls and lecture notes [13, 14]. While Otter.ai offers applications on different platforms including web, iOS and Android [15], this discussion will focus exclusively on the iOS application.

2.1.1 Retrieval Strategies

Otter.ai provides three primary mechanisms for retrieving information from transcripts: keyword-based search, LLM-powered conversational querying (Otter AI Chat), and LLM-Based Summarization, as illustrated in Figure 2.1 [12, 15].

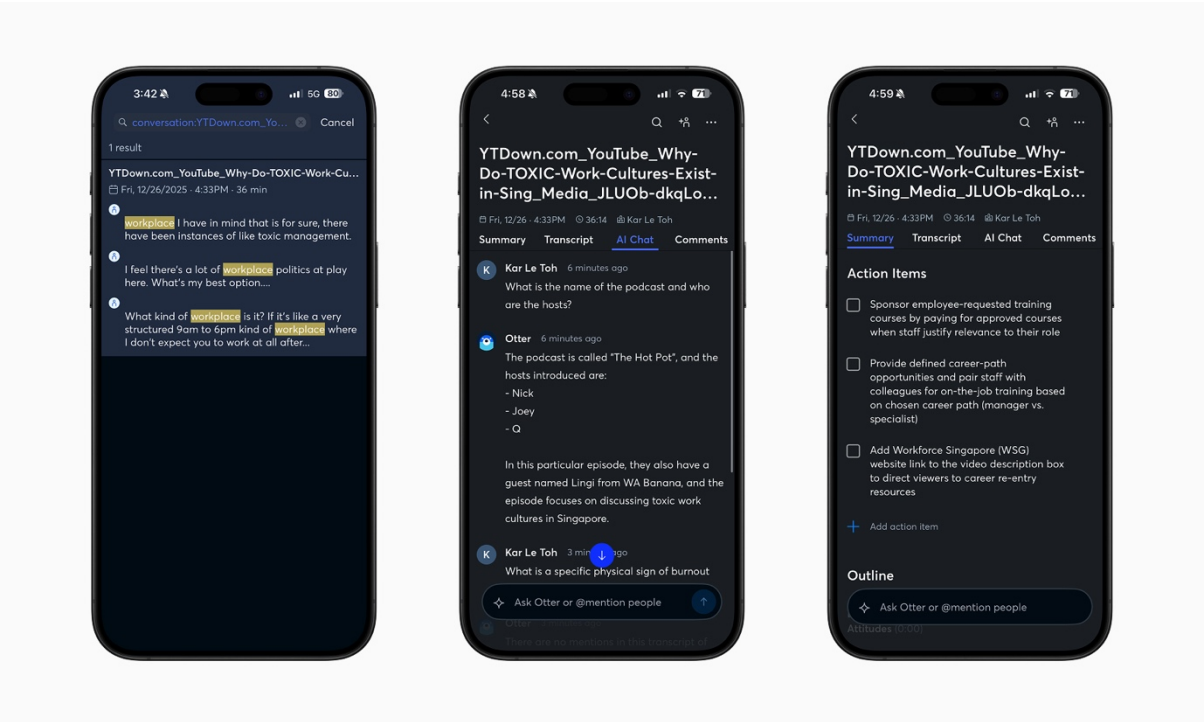

Figure 2.1: Otter.ai iOS application interface showing keyword search, AI chat, and summarization features

Keyword-Based Retrieval

Otter.ai supports keyword-based search over notes and transcripts [15]. Although Otter.ai does not publicly disclose the exact retrieval algorithm used, empirical testing suggests that the system employs a term-weighted ranking function such as TF-IDF or BM25. For example, queries containing additional discriminative terms such as "toxic workplace" versus "workplace" significantly alter result rankings in a manner consistent with BM25-style term frequency saturation and inverse document frequency weighting [16]. In addition, Otter.ai supports date-based filters over transcript metadata [15]. This feature substantially improves usability and demonstrates the value of combining structured metadata filtering with unstructured text search.

LLM-Based Conversational Retrieval (Otter AI Chat)

Otter AI Chat allows users to ask free-form natural language questions either across all transcripts or within a specific transcript. While Otter.ai does not explicitly disclose its backend architecture, its FAQ indicates that transcript content and prompts are transmitted to third-party services only when users actively interact with Otter AI Chat, strongly suggesting reliance on externally hosted LLMs from third-party providers [17].

Behavioral observations indicate that the global chat interface likely employs a retrieval step to narrow down relevant transcripts before passing context to the LLM, as responses often include links to specific conversations. This behavior is consistent with a RAG pipeline operating at the document level. In contrast, the per-transcript chat interface appears to process the entire transcript holistically, as responses rarely reference precise supporting segments or timestamps.

Notably, Otter AI Chat responses tend to be abstractive and conversational, favoring rephrased summaries over verbatim quotations. This design choice may be driven by prompt engineering rather than retrieval limitations; however, it also suggests limited emphasis on fine-grained, chunk-level semantic retrieval at query time.

LLM-Based Summarization

Beyond interactive retrieval, Otter.ai generates an extensive per-transcript summary using LLM-based processing [12]. This summary is structured into three layers: (i) a high-level overview of the meeting, (ii) extracted action items, and (iii) a hierarchical outline organized by timestamps, with further speaker-segmented summaries nested within each time interval. This multi-level summarization demonstrates effective use of LLMs to transform long-form transcripts into structured, navigable representations.

2.1.2 Accuracy and Latency Characteristics

From manual evaluation, Otter.ai's keyword-based search exhibits both high accuracy and negligible latency, consistent with expectations for traditional lexical retrieval systems. This makes keyword search the most reliable and responsive mechanism for precise information lookup. Otter AI Chat, when applied to transcripts up to approximately two hours in length, also demonstrates low perceived latency and strong answer accuracy. This performance is likely enabled by the use of large, cloud-hosted LLMs with sufficient context windows and optimized inference infrastructure.

2.1.3 Transparency of Retrieval and Reasoning

A notable limitation of Otter AI Chat is its lack of transparency. The system provides minimal visibility into which transcript segments support a given answer, and users cannot inspect or verify the retrieved evidence. This opacity limits user trust and makes error diagnosis difficult, particularly in professional or compliance-sensitive settings.

2.1.4 Discussion and Implications

Overall, Otter.ai exemplifies a feature-rich and well-integrated retrieval ecosystem for meeting transcripts. It combines global and local keyword search, conversational querying, and comprehensive summarization to support diverse user workflows. These capabilities collectively demonstrate that effective transcript systems must treat retrieval as a first-class design concern, rather than a secondary feature layered on top of transcription.

However, the apparent lack of fine-grained, chunk-level grounding and the absence of explicit evidence attribution indicate that Otter.ai's system prioritizes fluency and abstraction over verifiability. While this design choice may be acceptable for general meeting summarization, it introduces risks of hallucination and reduces user trust in scenarios where factual accuracy and traceability are critical. This limitation is particularly rele-

vant to Meadow9+, since it cannot rely on large cloud-hosted frontier models with extensive context windows and optimized inference pipelines. Instead, it employs a smaller, self-hosted LLM with stricter latency and context constraints. Consequently, passing full transcripts, or even large document-level contexts, into the model is neither scalable nor reliable.

These observations collectively imply that Meadow9+ must adopt a *more retrieval-centric and evidence-grounded design* than Otter.ai. Rather than treating retrieval as a preliminary filtering step, retrieval must be a first-class system component that operates at multiple granularities (document-level, chunk-level, and temporal level). Figure 2.1 illustrates the baseline chatbot architecture commonly observed in existing systems, where retrieval is shallow or opaque and tightly coupled to generation.

In contrast, Meadow9+ is designed to decouple retrieval from generation and to emphasize transparent, inspectable retrieval results. This includes (i) hybrid lexical–semantic retrieval to handle both exact terms and paraphrased queries, (ii) phonetic and fuzzy matching to mitigate transcription errors, and (iii) chunk-level evidence selection to reduce prompt length. By explicitly surfacing retrieved segments and their timestamps, Meadow9+ aims to improve both answer accuracy and user trust, which is an aspect largely absent in Otter.ai's conversational interface.

2.2 Claude Code

Recent advances in LLM–based agents have extended beyond conversational assistants into domain-specialized systems that tightly integrate retrieval, reasoning, and action. One notable example is Claude Code, a command-line interface (CLI) agent developed by Anthropic that brings LLM-driven reasoning directly into the software development workflow [18]. Claude Code is widely regarded as one of the most capable coding agents currently available, not solely due to model quality, but because of its system-level retrieval and context management design that enables effective reasoning over large, evolving codebases.

2.2.1 Retrieval Strategies

Claude Code adopts an agentic retrieval paradigm, where the LLM actively searches and retrieves relevant context from the environment at query time rather than relying on a precomputed embedding index or static vector store. Specifically, Claude Code is granted controlled read access to the local project directory and retrieves information by invoking standard developer tools such as file search and content matching when needed [19, 20]. Retrieved files or code snippets are then injected directly into the model's context window.

Importantly, retrieval is performed dynamically through tool calls that operate over the source-of-truth codebase rather than through a precomputed embedding index [19, 20]. This design avoids issues associated with stale embeddings, incomplete indexing, and semantic drift, ensuring that the retrieved context always reflects the most recent state

of the project.

Claude Code also supports persistent project-level memory via a lightweight mechanism: a configuration file (e.g., CLAUDE.md) that stores high-level instructions, architectural conventions, and recurring commands. This file is automatically injected into every session, acting as a form of long-term context without requiring a separate memory model or database [21].

2.2.2 Accuracy and Latency Characteristics

The agentic retrieval strategy in Claude Code introduces clear trade-offs between accuracy and latency. On the one hand, dynamically searching the codebase yields high retrieval precision, as the agent can iteratively refine its search queries and inspect exact source files relevant to the task [19]. This approach can improve task success rates in complex, multi-file coding scenarios compared to static retrieval pipelines.

On the other hand, retrieval incurs non-trivial overhead. Each file read and search operation increases prompt size and introduces runtime latency due to tool execution and additional token processing. This design typically leads to higher token usage and slower individual responses compared to systems that rely on pre-indexed embeddings, but the overhead can be acceptable given the resulting improvement in task accuracy and robustness.

To further improve accuracy on complex tasks, Claude supports an extended thinking mode in which the model spends more time deliberating before producing an answer [22]. While this increases latency, it can improve solution quality for tasks requiring multi-step reasoning or global code understanding.

2.2.3 Transparency of Retrieval and Reasoning

A defining characteristic of Claude Code is its emphasis on transparent retrieval and execution. Tool use is permission-gated, and any action that may modify the system requires user approval [19, 20]. This design allows developers to observe how context is retrieved and to intervene when necessary.

Claude Code also provides a "Plan Mode," in which the agent analyzes the codebase and proposes a sequence of actions without executing them. This separation of planning and execution improves trust and enables users to validate the agent's understanding before committing changes [18].

From a system-design perspective, Claude Code demonstrates that retrieval transparency is not only a safety feature but also a debugging aid, allowing users to reason about failures in retrieval or context selection.

2.2.4 Discussion and Implications

Although Claude Code operates in the domain of software engineering, its retrieval design offers several transferable insights for transcript-based systems such as Meadow9+.

First, Claude Code's avoidance of static embedding-based retrieval highlights a broader design principle: retrieval systems must align with the nature of the underlying data. Claude Code adopts a fully agentic retrieval approach precisely because of the characteristics of its target domain. Dynamic, query-time search over the source-of-truth codebase ensures that retrieved context remains up-to-date and semantically precise, avoiding issues associated with stale or incomplete indices.

In contrast, Meadow9+ operates over transcript data that is static once transcription is completed. While a full agentic retrieval paradigm is powerful, it is not strictly required in this setting. Additionally, practical system constraints must also be considered, as Meadow9+ employs a smaller self-hosted LLM with substantially weaker reasoning and tool-use capabilities compared to commercial frontier models used in systems such as Claude Code. Under these constraints, fully agentic retrieval may introduce performance bottlenecks, increased latency, or even unstable retrieval behavior, particularly when iterative reasoning and tool invocation are required.

These observations suggest that the optimal retrieval strategy for Meadow9+ lies in a hybrid approach, combining embedding-based semantic retrieval with symbolic techniques. Such a design leverages the static nature of transcript data to enable efficient indexing while remaining robust to transcription noise and lexical variation. At the same time, it avoids the computational and reasoning overheads associated with fully agentic retrieval, making it more suitable for deployment with resource-constrained, self-hosted LLMs.

Second, Claude Code's trade-off between accuracy and latency offers guidance for Meadow9+'s system design. Claude Code deliberately trades off latency and token efficiency in exchange for higher task accuracy. Each retrieval step incurs additional overhead due to tool invocation, file inspection, and prompt expansion. This trade-off is acceptable in developer workflows, where correctness is often prioritized over response time and where commercial frontier models with strong reasoning capabilities are available.

Meadow9+, however, operates under stricter latency and cost constraints. The system also relies on a self-hosted, smaller-scale LLM, which is significantly less capable than proprietary frontier models. Under such constraints, a fully agentic retrieval loop may introduce excessive latency and unstable performance. Moreover, iterative retrieval decisions place additional reasoning burden on the model itself, increasing the risk of suboptimal search strategies and error propagation.

These observations suggest that the retrieval strategy for Meadow9+ should be front-loaded into deterministic or weakly learned components, such as BM25 and dense embeddings, rather than delegated to the LLM. This reduces the reasoning load on the model and yields more predictable latency characteristics.

Finally, Claude Code's emphasis on transparent retrieval suggests an additional opportunity for Meadow9+. While Meadow9+ targets non-technical users, partial transparency,

such as highlighting which transcript segments were used to generate an answer, can improve user trust and perceived reliability. This is especially important in professional meeting contexts where incorrect attribution or hallucinated decisions can have serious consequences.

Incorporating lightweight transparency mechanisms aligns with Claude Code's philosophy that retrieval visibility is not merely a safety feature but a debugging and validation aid. Such features can also support iterative refinement of retrieval strategies during system evaluation.

2.3 Pre-Project Meadow9+

Meadow9+ is a mobile speech-to-text application developed by researchers from the NTU Speech and Language Laboratory, with initial development dating back to approximately 2021 (exact date to be confirmed) [23,24]. It is a product of ABAX AI, a company specializing in advanced speech recognition technologies tailored for enterprise and professional use cases [24, 25].

The platform is primarily designed for Singapore-based business professionals, with a strong emphasis on one-to-one meetings as its core usage scenario [23–25]. Nevertheless, Meadow9+ also supports a broader range of transcription contexts, including multi-party discussions and news audio recordings. A key strength of the system lies in its ability to handle Singaporean accents and domain-specific terminology, addressing a notable gap in the local speech recognition market where generic transcription systems often underperform.

While Meadow9+ demonstrates strong transcription quality, particularly in localized linguistic contexts, its post-transcription information retrieval and interaction capabilities remain limited. The following subsections analyze the retrieval and summarization mechanisms implemented in the pre-project version of Meadow9+, highlighting their strengths and limitations.

2.3.1 Retrieval Strategies

Meadow9+ provides two primary mechanisms for retrieving information from transcripts: (1) text-matching search and (2) LLM-based conversational querying.

Text-Matching Search

The system supports a text-matching search functionality at the global level, allowing users to search across conversation titles. At query time, the search matches user-provided keywords against stored conversation titles, returning transcripts whose metadata contains the queried terms. This design emphasizes simplicity and responsiveness, making it suitable for lightweight retrieval scenarios where meaningful titles are consistently available.

LLM-Based Conversational Retrieval

Meadow9+ integrates an LLM-powered chatbot that allows users to pose free-form natural language questions within a selected transcript [26]. For privacy and data governance considerations, the chatbot relies on a self-hosted LLM.

In the pre-project system, the chatbot passes the full transcript directly into the LLM as input context at query time, without an intermediate retrieval or context selection stage. Responses generated by the system are primarily abstractive and conversational in nature, focusing on synthesized summaries rather than direct quotations or fine-grained references to specific transcript segments.

LLM-Based Summarization

In addition to interactive querying, Meadow9+ generates a per-transcript summary using LLM-based processing [26]. This summary provides a condensed overview of the meeting content and is generated independently of user queries. The summarization output is static and intended for post-meeting review rather than targeted information retrieval. It does not support interaction, filtering, or alignment with subsequent user questions.

2.3.2 Accuracy and Latency Characteristics

Based on manual evaluation, Meadow9+'s text-matching search demonstrates high accuracy and minimal latency under ideal conditions. When relevant metadata is present, search results are returned promptly.

In contrast, the chatbot exhibits high perceived latency and weak answer accuracy when applied to transcripts exceeding approximately 1 hour in length. These performance issues are likely attributable to two factors: (1) the use of a relatively small self-hosted LLM with limited reasoning capacity compared to commercial frontier models, and (2) inefficient utilization of the model's context window due to the absence of retrieval or context selection mechanisms.

2.3.3 Transparency of Retrieval and Reasoning

The pre-project Meadow9+ chatbot presents users with a single generated response per query. The system does not provide explicit indicators of which transcript segments were most relevant to the answer, nor does it surface supporting excerpts or references from the transcript. As a result, users interact with the chatbot as a black-box conversational interface, without visibility into intermediate reasoning or evidence selection processes.

2.3.4 Discussion and Implications

The pre-project Meadow9+ system demonstrates a clear separation between transcription quality and post-transcription interaction capabilities. While the platform is optimized for accurate speech recognition in localized contexts, its retrieval and interaction mechanisms are comparatively lightweight and monolithic, exhibiting several structural limitations that impact effectiveness, scalability, and user experience.

For text-matching search, the reliance on transcript-level metadata significantly constrains retrieval coverage. Because conversation titles and descriptions are optional, many transcripts lack sufficiently descriptive metadata, reducing the practical utility of this search mechanism. Moreover, the absence of keyword-based searching within transcript content prevents users from retrieving specific terms, entities, or discussion points embedded in the conversation itself. The system also does not support natural language or temporal constraints (e.g., "last Friday" or "earlier in the meeting"), which are increasingly expected in modern retrieval interfaces.

The LLM-based conversational retrieval approach addresses some expressiveness limitations by allowing free-form queries; however, its design introduces different trade-offs. Passing entire transcripts directly to the LLM leads to high inference latency for long-duration meetings and exacerbates the lost-in-the-middle problem. Additionally, the abstractive nature of responses, combined with the lack of explicit evidence grounding, reduces answer verifiability and makes systematic error analysis difficult.

Collectively, these observations highlight a fundamental architectural gap in the pre-project Meadow9+ system: the absence of an explicit, fine-grained retrieval layer that mediates between raw transcripts and language model generation. This gap motivates the adoption of a structured RAG framework in the proposed system. By decoupling retrieval from generation and incorporating hybrid retrieval strategies tailored to noisy speech-to-text data, the redesigned system aims to improve latency, accuracy, transparency, and scalability while preserving conversational usability.

Chapter 3

Proposed System Architecture

3.1 System Overview

The proposed system extends the Meadow9+ transcript assistant from a chatbot-centric design into a hybrid retrieval system. Instead of relying on a single interaction mode that faces scalability and latency limitations, the redesigned system supports three complementary capabilities over the same transcript corpus: a summarization pipeline for high-level transcript understanding, a retrieval-augmented generation (RAG) pipeline for conversational question answering, and a keyword-based search interface for direct snippet retrieval. [4]

To minimise disruption to the existing Meadow9+ infrastructure, the new retrieval system is deployed as a separate retrieval server that interacts with the current Meadow9+ backend server. This separation reduces the extent of modifications required on the production backend, while enabling the possibility of controlled A/B testing in production to compare the performance of the existing chatbot approach against the new retrieval pipeline proposed in this project.

At a high level, the system operates in two distinct phases: an indexing phase and a querying phase.

3.2 Indexing Phase

During the indexing phase, when a user submits a new recording and a corresponding transcription is generated, the transcript continues to be stored in the existing Meadow9+ database as per the current implementation. In addition, the same transcript is forwarded to the Retrieval Server, as illustrated in Figure 3.1.

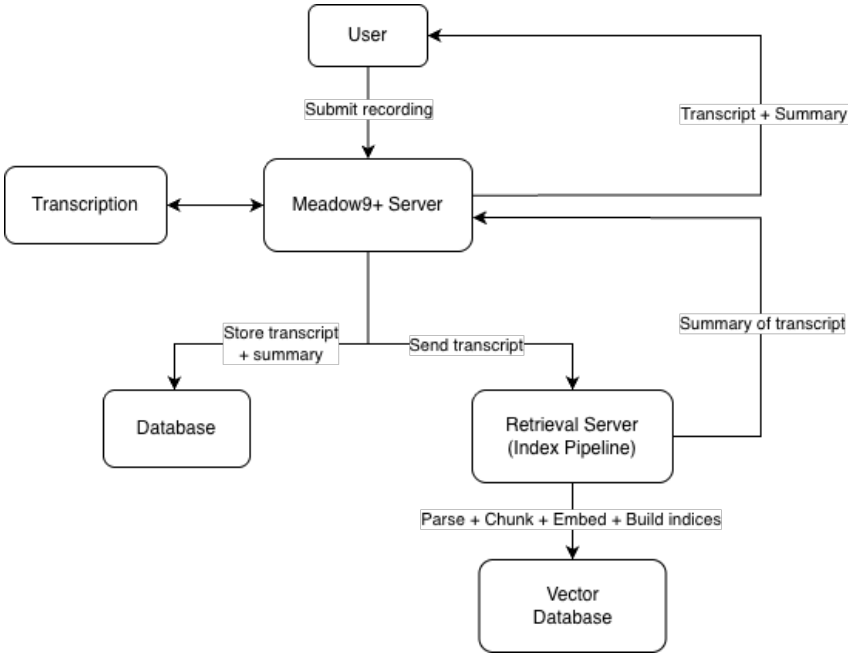

Figure 3.1: System integration: transcript forwarding, summary return, and retrieval indexing

The Retrieval Server performs two tasks in parallel. First, it generates a raw transcript summary and returns it to the Meadow9+ Server. This summary provides high-level abstractions of the transcript, including overview, topic, outline, segment, action, and entity-level information. The Meadow9+ Server can then store this returned summary in the database and return it to the user as a high-level summary of the transcript.

Second, the Retrieval Server performs the processing, chunking, and embedding stages required to prepare the transcript for later retrieval. Figure 3.2 shows this high-level indexing pipeline, where the raw transcript is processed and transformed into indexed representations that can later support retrieval-augmented question answering and search.

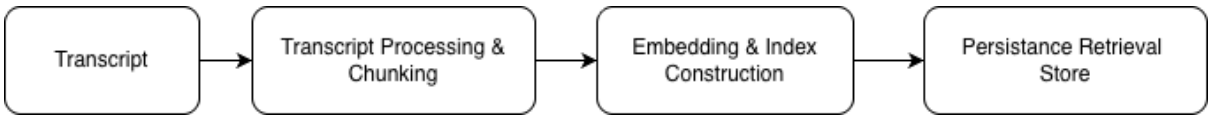

Figure 3.2: Indexing phase: transcript processing, chunking, and embedding stages

3.3 Querying Phase

During querying, user questions are transformed into embeddings, used to retrieve a small set of relevant transcript segments, and supplied to the LLM as contextual grounding for response generation. This query workflow is depicted in Figure 3.3, with the complete RAG pipeline architecture shown in Figure 3.4. [4]

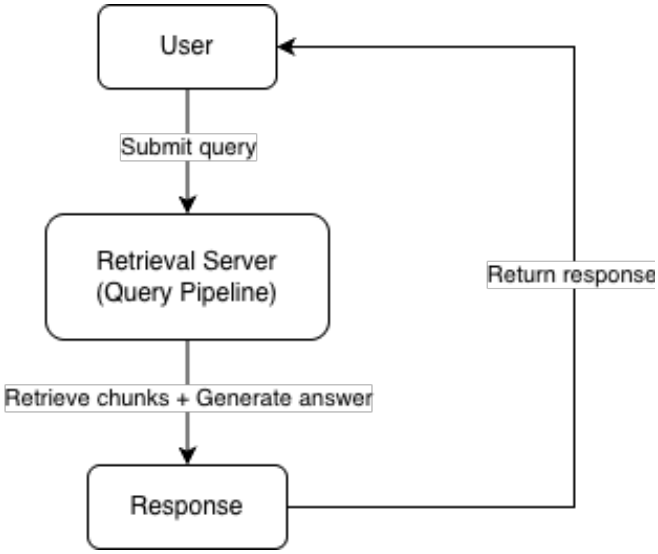

Figure 3.3: Querying phase: retrieval and response generation

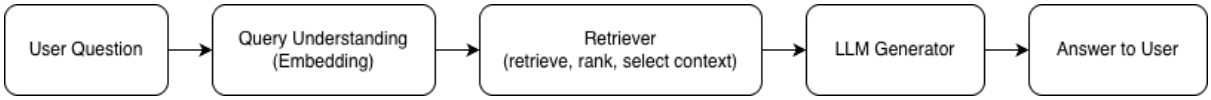

Figure 3.4: Querying phase: high-level query-to-response architecture

3.4 Hybrid Retrieval System

This architecture supports a Hybrid Retrieval System consisting of three complementary components. The first component is the Summarization Pipeline, which provides users with high-level transcript understanding by generating transcript summaries from newly uploaded transcripts. This pipeline is also crucial because the produced summary works hand-in-hand with the RAG pipeline to support more vague, high-level questions that benefit from transcript-level framing rather than only fine-grained evidence retrieval. The second component is the RAG Pipeline, which supports conversational question answering by retrieving relevant transcript evidence and grounding LLM responses in that retrieved context. The third component is Keyword-Based Search, which enables faster direct retrieval for keyword-based, date-based, and speaker-based lookup tasks.

Together, these three components enable the system to support both high-level understanding and fine-grained retrieval over long transcripts. The detailed implementation of these components will be discussed in Chapter 4.

3.5 Evaluation Architecture

The current evaluation architecture focuses on the conversational QA path of the RAG subsystem. At a high level, the system evaluates retrieval and answer generation using

a dedicated question–answer dataset derived from transcript data, then runs batch evaluation to measure how effectively the system retrieves relevant evidence and produces grounded answers.

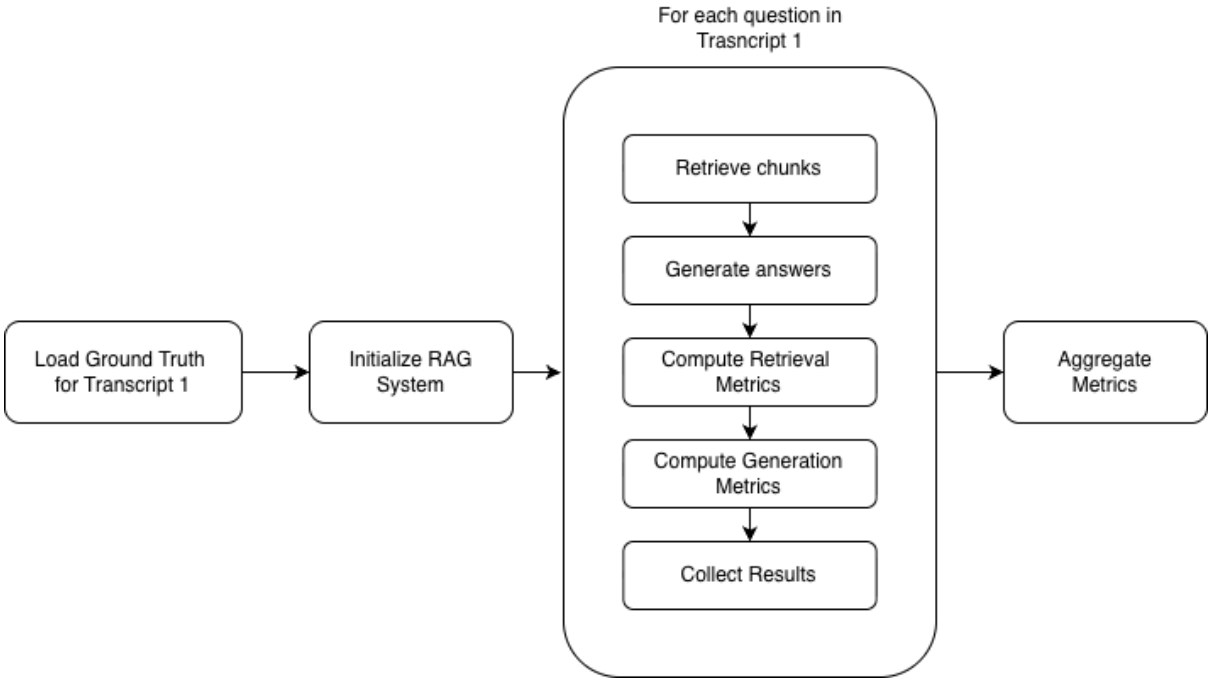

Figure 3.5: Evaluation pipeline for the RAG question-answering subsystem

The evaluation outputs include retrieval metrics, generation metrics, and speed benchmarking. Retrieval quality is assessed using ranking-based measures such as hit rate and mean reciprocal rank (MRR), while generation quality is assessed using lexical and semantic correctness measures. [9, 27] In addition, the system benchmarks the RAG-based QA subsystem against a pure-LLM baseline to compare latency and overall response efficiency across different transcript lengths, as shown in Figure 3.6.

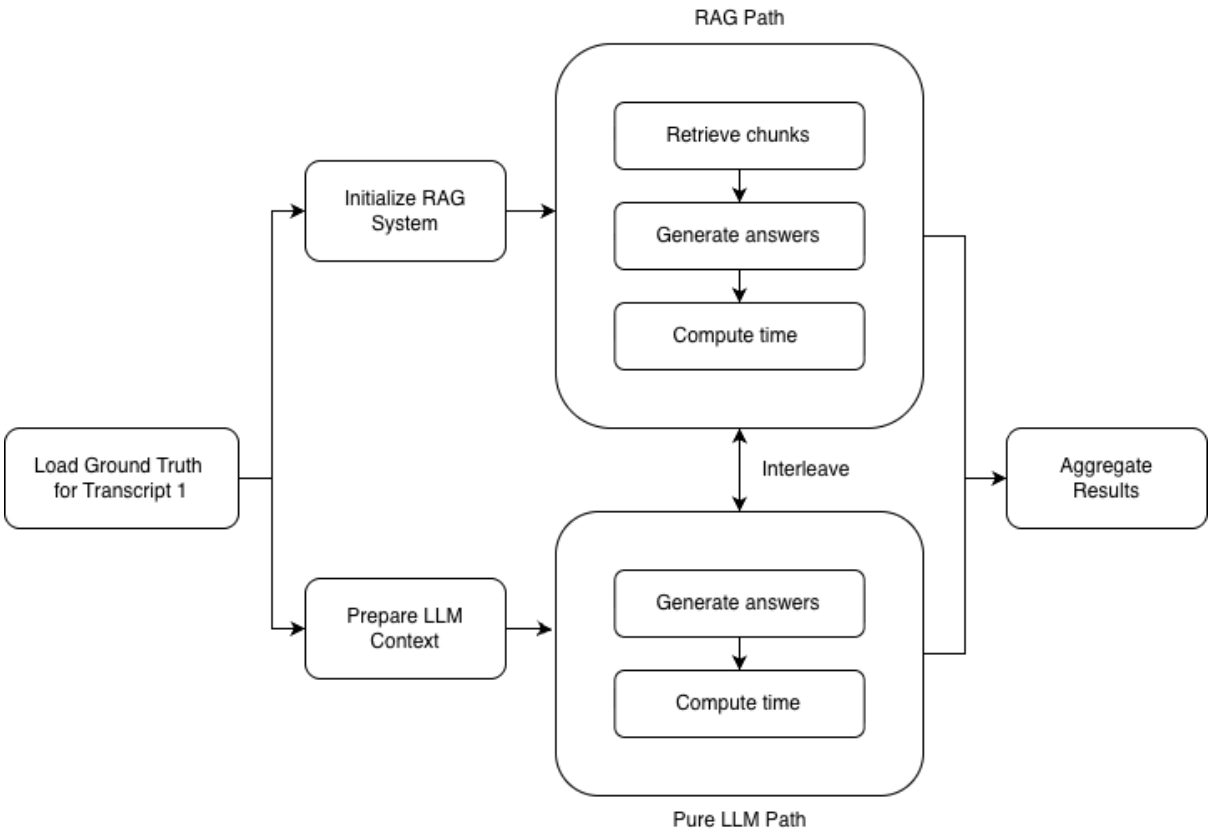

Figure 3.6: Benchmarking: RAG-based QA subsystem vs. pure LLM baseline

This evaluation architecture focuses on the RAG pipeline, as this is the subsystem that can be directly compared against the existing chatbot in terms of performance, scalability, and latency. The summarization pipeline and keyword-based search interface are presented as implemented capabilities of the proposed system, but they are not the primary focus of the comparative evaluation.

Chapter 4

Implementation and Progress to Date

This chapter presents the implementation of the hybrid retrieval system developed for the Meadow9+ transcription platform. Rather than centering the project on a single RAG chatbot pipeline, the final system comprises three coordinated subsystems built on shared transcript processing and retrieval infrastructure: a summarization pipeline, a RAG pipeline for conversational question answering, and a keyword-based search interface for direct snippet retrieval.

4.1 External Resources Incorporated

The hybrid retrieval system integrates multiple external libraries and services to support transcript processing, summary generation, retrieval, reranking, API serving, and evaluation. These resources are summarized in Table 4.1.

4.1.1 Core Framework Libraries

The system is built on the LangChain ecosystem, which provides core abstractions for document handling, prompt construction, and output parsing. The web API layer uses FastAPI as the REST framework with Pydantic for schema validation and serialization, enabling type-safe request and response handling across the implemented endpoints. [28–32]

4.1.2 Models Integration

The shared retrieval foundation uses a self-hosted jina-embeddings-v3-f16 model for dense retrieval and a bge-reranker-v2-m3-FP16 model for reranking retrieved candidates. The local Qwen3-14B-Q4 model is used both for grounded answer generation in the RAG subsystem and for long-transcript summarization. For evaluation pipelines, DeepSeek is used via the DeepSeek API to generate ground-truth answers and semantic judgments. All local models are hosted behind llama.cpp server, which provides OpenAI-compatible API endpoints for model serving. [33–37]

4.1.3 Other Libraries

Additional libraries support hybrid retrieval, metadata parsing, numerical analysis, and development workflows. In particular, rank-bm25, metaphone, rapidfuzz, and wordfreq

support the lexical and ASR-robust retrieval signals used by the RAG and search subsystems, while dateparser enables natural-language date filtering in the search bar. [38–50]

Category	Key Libraries	Primary Purpose
Framework	LangChain, FastAPI, Pydantic	Prompt orchestration, document abstraction, and REST API serving
Vector Store	Chroma	Persistent embedding storage and similarity search
Retrieval	rank-bm25, metaphone, rapidfuzz, wordfreq	Hybrid retrieval with keyword, phonetic, and fuzzy signals
LLM Integration	langchain-openai, llama.cpp server	OpenAI-compatible access to local and external models
Data Processing	dateparser, pandas, numpy, matplotlib	Date parsing, data analysis, and evaluation support
Development	pytest, requests	Testing and service communication

Table 4.1: Third-party libraries and frameworks used across the hybrid retrieval system

4.2 Summarization Pipeline Implementation

The summarization pipeline was added to support high-level transcript understanding and to provide transcript-level retrieval artefacts for broad or vague downstream queries. A single full-context LLM call is not reliable for long transcripts, so the implementation adopts a hierarchical multi-stage design in which summarization is decomposed into chunking, per-chunk extraction, and global synthesis.

4.2.1 Transcript Chunking for Summarization

The summarization path first splits the transcript into token-counted chunks rather than relying on a simple character limit. Chunk boundaries are aligned as far as possible with speaker turns so that local summaries preserve conversational flow, participant structure, and topic boundaries. Small overlaps are carried across adjacent chunks to reduce information loss at chunk boundaries, which is particularly important for discussions that span multiple turns or contain deferred references.

This chunking strategy differs in purpose from transcript chunking for retrieval. In the summarization subsystem, chunking exists to keep each extraction request within the local model's practical input budget while preserving enough local coherence for reliable structured extraction.

4.2.2 Per-Chunk Structured Extraction

Each chunk is then processed asynchronously by the local Qwen model to generate a structured JSON-style chunk summary. The implementation uses a structured-output path when supported by the serving stack and falls back to prompt-based JSON parsing if the server rejects schema-enforced output. This fallback is important because summarization is expected to operate robustly across different llama.cpp builds and local deployment configurations.

The chunk extraction stage runs with bounded concurrency to avoid saturating the locally hosted LLM service. Each chunk summary captures local discussion points, entities, and action items, forming an intermediate representation that is easier to merge than raw transcript text.

Placeholder: summarization pipeline implementation view.
The figure should show token-aware chunking, bounded-concurrency chunk extraction, structured-output with fallback parsing, and hierarchical merge into the final transcript summary JSON.

Figure 4.1: Placeholder for the implemented summarization pipeline

4.2.3 Hierarchical Merge and Canonical Summary Schema

After chunk extraction, the intermediate summaries are merged into a canonical transcript-level summary. If all extracted summaries fit into a single merge call, the system performs one synthesis step. If the combined payload exceeds the available context budget, the implementation falls back to hierarchical merging, where batches of chunk summaries are merged into intermediate summaries before a final pass combines them into one output.

The resulting canonical summary is stored in structured JSON form. Its fields capture transcript-level framing information such as participants, conversation type, purpose, overview, key topics, detailed topics, recurring themes, outline sections, notable segments, action items, and entities. This schema is richer than a simple free-text abstract and is designed to support both user-facing summarization and downstream retrieval over specific summary sections.

4.2.4 Summary Embedding Documents

The summarization subsystem does not stop at producing one canonical summary JSON file. During ingestion, the system also transforms the summary into multiple semantically focused retrieval documents, such as overview, topic, outline, segment, action, entity, and fallback miscellaneous documents. Each of these derived documents is tagged with summary-specific metadata and embedded into the same retrieval infrastructure used for transcript chunks.

This design solves two problems simultaneously. First, it avoids representing the entire summary as one monolithic vector, which would be too coarse for topic-level matching. Second, it allows broad or transcript-level queries to retrieve focused summary sections instead of only low-level transcript snippets. These summary documents therefore function as retrieval artefacts, not merely display data.

Placeholder: summary embedding document generation.
The figure should show a canonical summary JSON being decomposed into overview, topic, outline, segment, action, entity, and miscellaneous summary documents, then embedded and stored alongside transcript chunks.

Figure 4.2: Placeholder for summary embedding document generation

4.3 RAG Pipeline Implementation

The RAG pipeline remains the primary subsystem for conversational question answering. However, in the current implementation it should be understood as one consumer of the shared hybrid retrieval foundation rather than the entire system itself. Its role is to transform a user question into a grounded answer by retrieving relevant evidence, assembling context, and invoking the LLM under explicit grounding constraints. [4]

4.3.1 Transcript Chunking Implementation

4.3.1.1 SRT Parsing

The SRT parser converts subtitle-format transcripts into structured Python dictionaries containing text, timestamps, sequence information, and speaker metadata. Figure 4.3 shows the input SRT structure, while Figure 4.4 shows the parsed output representation.

```
1
00:00:00,640 --> 00:00:15,140
Hi, how are you today?

2
00:00:15,140 --> 00:01:31,240
>> Speaker: Read this aloud for me...
```

Figure 4.3: SRT input format with sequence numbers and timestamps

```
{
    'sequence': 1,
    'start_time': '00:00:00,640',
    'end_time': '00:00:15,140',
    'text': 'Hi, how are you today?',
    'speaker': None  # or 'Speaker' if ">>" prefix detected
}
```

Figure 4.4: Parsed SRT structure with speaker metadata extraction

Speaker detection handles both simple speaker markers and explicit speaker-name prefixes. Speaker names are extracted and stored separately from the content text so that later retrieval and filtering stages can use speaker metadata without polluting the searchable transcript content.

4.3.1.2 Token-Based Merging

For retrieval, the system merges small subtitle units into larger chunks using token-counted boundaries. This is more reliable than character-based estimation because it aligns chunk size management with the actual tokenizer limits used by the embedding and generation models. The merging strategy also preserves natural subtitle boundaries, which keeps timestamp alignment and speaker attribution intact for downstream citation and answer grounding.

```
1. For each chunk:
   a. Count tokens using TokenCounter
   b. If chunk < MIN_CHUNK_SIZE (25 tokens):
      - Add to accumulator
      - If accumulator >= TARGET_MERGED_SIZE (128 tokens):
        * Create merged chunk
        * Update timestamps (earliest start, latest end)
        * Track merged_count
   c. Else:
      - Flush accumulator if non-empty
      - Emit chunk as-is

2. Flush remaining accumulator
```

Figure 4.5: Token-based merging algorithm for chunk size optimization

4.3.2 Embedding Generation and Vector Storage

4.3.2.1 Global Unified Index Architecture

All transcript chunks are stored in a single global vector store instead of separate per-transcript indices. This simplifies index management, enables cross-transcript search, and allows transcript scoping through metadata filters rather than separate collections.

4.3.2.2 Embedding Service

The embedding service is deployed separately from the answer-generation LLM and reranker service. This separation enables independent scaling, resource allocation, and model replacement across the system's inference components.

4.3.2.3 Transcript Processing

Figure 4.6 shows the document processing pipeline for a single transcript. The pipeline parses the SRT file, constructs LangChain Document objects, and attaches transcript-specific metadata such as transcript_id, timestamps, chunk identifiers, and speaker information.

```python
def process_transcript(self, transcript_path: str) -> List[Document]:

    filename = os.path.basename(transcript_path)
    print(f"Processing {filename}...")

    # Parse SRT file into chunks
    chunks = parse_srt_chunks(transcript_path)
    print(f"  Parsed {len(chunks)} SRT entries")

    # Create documents (with merging if enabled)
    documents = create_documents_from_srt_chunks(
        chunks,
        file_path=filename
    )

    # Add transcript_id metadata (cleaned filename without extension)
    transcript_id = filename.replace('.srt', '').replace('.', '_')
    for doc in documents:
        doc.metadata['transcript_id'] = transcript_id

    print(f"  Created {len(documents)} document chunks")
    return documents
```

Figure 4.6: Document processing pipeline

4.3.2.4 Batch Embedding Generation and Vector Store Creation

Embedding generation is one of the most computationally expensive steps in the indexing workflow. The implementation therefore relies on batched embedding requests through the vector-store integration layer rather than one document at a time. Figure 4.7 shows vector-store creation from documents, while Figure 4.8 shows incremental document addition for new transcripts. [39, 40]

```python
# Create vector store
print(f"  Creating Chroma vector store (this will take a while)...")

vector_store = Chroma.from_documents(
    documents=all_documents,
    embedding=self.embeddings,
    collection_name=self.collection_name,
    persist_directory=self.global_store_path
)

print("  ✓ Vector store created and persisted")
```

Figure 4.7: Vector store creation from documents

```python
print(f"\n  Adding {len(all_new_documents)} new chunks to vector store...")

# Pass explicit IDs to enable upsert behavior (prevent duplicates)
ids = [doc.id for doc in all_new_documents if hasattr(doc, 'id') and doc.id]
if ids and len(ids) == len(all_new_documents):
    vector_store.add_documents(all_new_documents, ids=ids)
else:
    vector_store.add_documents(all_new_documents)

print("  ✓ New documents added to vector store")
```

Figure 4.8: Incremental document addition

4.3.2.5 ChromaDB Storage and Persistence

Chroma persists embeddings, metadata, and collection configuration within a local storage directory. Figure 4.9 illustrates how the vector store, BM25 index, and phonetic index are co-located in the same storage hierarchy, simplifying backup and deployment. [40]

```
vector_stores/global_combined/
├── chroma.sqlite3              # Main vector database (embeddings + metadata)
├── phonetic_index.db           # Phonetic search index (SQLite)
└── bm25_index/
    └── bm25_index.pkl          # Serialized BM25 index
```

Figure 4.9: ChromaDB storage directory structure

This shared storage layer now also coexists with the summary-derived retrieval documents produced by the summarization subsystem, enabling broad and detailed retrieval over the same transcript corpus.

4.3.3 Multi-Stage Retrieval Process

The RAG retrieval workflow follows a multi-stage refinement process in which broad candidate recall is achieved first, followed by rank fusion and precision-oriented reranking.

4.3.3.1 Stage 1: Hybrid Search

The first stage executes four complementary retrieval methods in parallel: vector search, BM25 keyword search, phonetic search, and fuzzy matching. [16, 51, 52] The goal is not to force one retrieval algorithm to handle all transcript-search conditions, but to combine signals that are robust to semantic variation, exact term matching, ASR noise, and typographical variation.

Vector Search (Semantic)

Vector search retrieves semantically related chunks using dense embeddings generated by the Jina model. Because the query and documents are embedded into the same vector space, the system can retrieve relevant chunks even when the query is phrased differently from the transcript text. [33]

```python
def run_semantic_search(
    vector_retriever,
    query: str,
    k: int,
    chroma_filter: Optional[Dict] = None
) -> List[Document]:

    if chroma_filter:
        return vector_retriever.vectorstore.similarity_search(
            query, k=k, filter=chroma_filter
        )
    else:
        return vector_retriever.invoke(query)
```

Figure 4.10: Vector search core function showing semantic retrieval with optional metadata filtering

BM25 Search (Keyword)

BM25 provides exact lexical matching and is particularly effective for proper nouns, product names, acronyms, and rare technical terms that must appear verbatim in the retrieved evidence. [16, 41] The implementation uses a lightweight tokenization strategy and supports transcript-scoped filtering.

```python
@staticmethod
def _default_tokenizer(text: str) -> List[str]:

    return text.lower().split()
```

Figure 4.11: BM25 tokenization function using lowercase split

```python
def __init__(
    self,
    documents: List[Document],
    tokenizer: Optional[Callable[[str], List[str]]] = None
):

    if not documents:
        raise ValueError("Cannot create BM25 retriever with empty document list")

    self.documents = documents
    self.tokenizer = tokenizer or self._default_tokenizer

    # Tokenize all documents for BM25 indexing
    print(f"Tokenizing {len(documents)} documents for BM25 indexing...")
    tokenized_corpus = [
        self.tokenizer(doc.page_content)
        for doc in documents
    ]

    # Build BM25 index
    print("Building BM25 index...")
    self.bm25 = BM25Okapi(tokenized_corpus)
    print(f"✓ BM25 index created with {len(documents)} documents")
```

Figure 4.12: BM25 index initialization with corpus tokenization

```python
# Tokenize query
tokenized_query = self.tokenizer(query)

if filter_value:
    # Normalize filter_value to list for consistent handling
    if isinstance(filter_value, str):
        filter_values = [filter_value]
    else:
        filter_values = filter_value

    # Filter documents by specified field before ranking
    filtered_indices = [
        i for i, doc in enumerate(self.documents)
        if doc.metadata.get(filter_field) in filter_values
    ]

    if not filtered_indices:
        # No documents match the filter
        return []

    # Get BM25 scores efficiently for only the subset of documents
    # get_batch_scores returns a list of scores corresponding to doc_ids
    scores = self.bm25.get_batch_scores(tokenized_query, filtered_indices)

    # Zip indices and scores together
    filtered_scores = list(zip(filtered_indices, scores))
```

Figure 4.13: BM25 query tokenization and metadata filtering

Phonetic Search (ASR Error Handling)

Phonetic search uses Double Metaphone encoding to improve recall when ASR errors produce sound-alike spellings, especially for proper nouns and technical terms. [42, 51]

```python
def tokenize_and_encode(self, text: str) -> List[Tuple[str, str]]:

    # Lowercase and extract words (alphabetic only)
    words = re.findall(r'[a-zA-Z]+', text.lower())

    # Filter short words (< 3 chars) - they're too common for phonetic matching
    words = [w for w in words if len(w) >= 3]
```

Figure 4.14: Phonetic word tokenization and filtering

```python
    # Compute phonetic codes
    phonetic_pairs = []
    seen = set()

    for word in words:
        # Get primary phonetic code (ignore alternate)
        code, _ = doublemetaphone(word)

        if code and (code, word) not in seen:
            phonetic_pairs.append((code, word))
            seen.add((code, word))

    return phonetic_pairs
```

Figure 4.15: Double Metaphone phonetic encoding

```
"Samsung" → "SMSNK"
"Samson" → "SMSN"
```

Figure 4.16: Phonetic encoding examples for homophone matching

```
# Build query with optional transcript filter
if transcript_filters:
    # With filter (single or multiple transcripts)
    code_placeholders = ','.join('?' * len(query_codes))
    transcript_placeholders = ','.join('?' * len(transcript_filters))
    sql = f"""
        SELECT chunk_id, COUNT(*) as overlap_count
        FROM phonetic_codes
        WHERE phonetic_code IN ({code_placeholders})
          AND transcript_id IN ({transcript_placeholders})
        GROUP BY chunk_id
        ORDER BY overlap_count DESC
    """
    params = query_codes + transcript_filters
```

Figure 4.17: Phonetic retrieval SQL query with transcript filtering

```
def run_phonetic_search(
    phonetic_indexer,
    query: str,
    vector_store,
    k: int,
    transcript_filters: Optional[List[str]] = None
) -> List[Document]:

    # Get phonetic matches from indexer
    phonetic_results = phonetic_indexer.retrieve(
        query,
        transcript_filters=transcript_filters,
        top_k=k
    )

    # Convert to scores dict
    phonetic_scores = {chunk_id: count for chunk_id, count in phonetic_results}

    # Build chroma filter for document fetching
    chroma_filter = None
    if transcript_filters:
        if len(transcript_filters) == 1:
            chroma_filter = {"transcript_id": {"$eq": transcript_filters[0]}}
        else:
            chroma_filter = {"transcript_id": {"$in": transcript_filters}}

    # Convert scores to ranked document list
    return _convert_scores_to_ranked_list(
        phonetic_scores,
        vector_store,
        chroma_filter
    )
```

Figure 4.18: Phonetic score conversion and document retrieval

Fuzzy Matching (Rare Token Typos)

Fuzzy matching complements phonetic search by handling character-level perturbations

through edit-distance similarity. To control computational cost, the implementation applies fuzzy matching selectively to rare tokens detected using Zipf frequency from the wordfreq library. [43, 44, 52]

```python
def is_rare_token(self, token: str) -> bool:

    # Get Zipf frequency (log10 of occurrences per billion words)
    # 0.0 means not in dictionary (infinite rarity)
    freq = zipf_frequency(token, 'en')

    # Threshold 4.0 corresponds to 10^4 per billion (1 in 100,000)
    # Common words > 4.0, Rare words < 4.0
    return freq < 4.0
```

Figure 4.19: Rare token detection using Zipf frequency

```python
def extract_rare_tokens(self, query: str) -> List[str]:

    # Tokenize (extract alphanumeric words)
    tokens = re.findall(r'\w+', query)

    # Filter to rare tokens
    rare_tokens = [t for t in tokens if self.is_rare_token(t)]

    return rare_tokens
```

Figure 4.20: Extracting rare tokens from query

```python
def compute_fuzzy_scores(
    self,
    query: str,
    candidate_pool: List[Document],
    top_k: Optional[int] = None
) -> Dict[int, float]:

    # Extract rare tokens from query
    rare_tokens = self.extract_rare_tokens(query)

    if not rare_tokens:
        return {}

    # Compute scores
    scores = {}
```

Figure 4.21: Fuzzy scoring algorithm: setup and initialization

```python
for doc in candidate_pool:
    chunk_id = doc.metadata.get('chunk_id')
    if chunk_id is None:
        continue

    doc_tokens = self._tokenize_document(doc)
    if not doc_tokens:
        continue

    # Compute best match score
    max_score = 0.0
    for query_token in rare_tokens:
        for doc_token in doc_tokens:
            # Compute edit-distance similarity using RapidFuzz
            score = fuzz.ratio(
                query_token.lower(),
                doc_token.lower()
            ) / 100.0  # Normalize to 0-1 scale

            max_score = max(max_score, score)

            # Early exit if we found a near-perfect match
            if max_score >= 0.95:
                break

        if max_score >= 0.95:
            break

    # Only include if above threshold
    if max_score >= self.threshold:
        scores[chunk_id] = max_score
```

Figure 4.22: Fuzzy scoring algorithm: nested loop with RapidFuzz ratio

```python
# Sort by score and limit to top_k
if top_k is not None and len(scores) > top_k:
    sorted_items = sorted(
        scores.items(),
        key=lambda x: x[1],
        reverse=True
    )[:top_k]
    scores = dict(sorted_items)

return scores
```

Figure 4.23: Fuzzy top-k selection and ranking

Integration: Parallel Multi-Signal Retrieval

These four methods are executed in parallel because they operate independently over the same indexed corpus. Their outputs are later fused into one ranked candidate pool. Table 4.2 summarizes their different roles in the RAG subsystem.

Search Method	Algorithm	Index Type	Scoring Metric	Top-K	Best For
Vector Search	Cosine similarity over embeddings	Dense vectors (Chroma)	Embedding distance	10	Semantic similarity, paraphrasing
BM25 Search	Probabilistic TF-IDF ranking	Inverted index	BM25 score	25	Keyword matching, proper nouns
Phonetic Search	Double Metaphone encoding	SQLite phonetic index	Overlap count	10	ASR errors, homophones
Fuzzy Matching	Edit-distance (RapidFuzz)	No index (full scan)	Fuzzy ratio	10	Typos, spelling variations

Table 4.2: Search method characteristics in the hybrid retrieval stage used by the RAG subsystem

4.3.3.2 Stage 2: Reciprocal Rank Fusion (RRF)

The ranked outputs of the four Stage 1 retrievers are combined using Reciprocal Rank Fusion. RRF normalises heterogeneous ranking outputs and rewards chunks that consistently rank well across multiple signals, making it a practical and robust fusion strategy for noisy transcript retrieval. [10]

```
score(d) = Σ weight_i * (1 / (k + rank_i(d)))

where:
  k = 60 (HYBRID_FUSION_K constant from literature)
  weight_i = search method weight (VECTOR_WEIGHT, BM25_WEIGHT, PHONETIC_WEIGHT, FUZZY_WEIGHT)
  rank_i(d) = rank of document d in search method i (1-indexed)

Configured weights:
  VECTOR_WEIGHT = 1.0    (primary signal)
  BM25_WEIGHT = 1.0      (primary signal)
  PHONETIC_WEIGHT = 0.2  (supplementary signal)
  FUZZY_WEIGHT = 0.2     (supplementary signal)
```

Figure 4.24: Reciprocal Rank Fusion weights for the hybrid retrieval signals

4.3.3.3 Stage 3: Cross-Encoder Reranking

After fusion, a cross-encoder reranker re-scores the candidate chunks for final precision-focused ranking. [11] This stage is more computationally expensive than the initial retrieval signals, so it is applied only to a relatively small candidate pool.

```
{
  "results": [
    {"index": 2, "relevance_score": 0.9523},
    {"index": 0, "relevance_score": 0.8876},
    {"index": 5, "relevance_score": 0.7234},
    {"index": 1, "relevance_score": 0.6891}
  ]
}
```

Figure 4.25: Expected reranker response format

```
# Map back to original documents
reranked_docs = []
for item in ranked_results:
    idx = item["index"]
    if 0 <= idx < len(documents):
        # Optionally add relevance score to metadata
        doc = documents[idx]
        # Create a copy with updated metadata
        reranked_doc = Document(
            page_content=doc.page_content,
            metadata={
                **doc.metadata,
                "rerank_score": item.get("relevance_score", 0)
            }
        )
        reranked_docs.append(reranked_doc)
```

Figure 4.26: Mapping reranked results back to the original documents

```
# Enforce top_n slicing (in case server ignores the parameter)
if top_n is not None:
    reranked_docs = reranked_docs[:top_n]

return reranked_docs
```

Figure 4.27: Top-n selection after reranking

4.3.4 Context Assembly and Prompt Construction

After retrieval and reranking, the selected evidence must be transformed into an LLM-ready prompt. This stage formats retrieved chunks, preserves citation metadata, injects route-aware context such as summary-derived transcript framing where needed, and then constructs a chat prompt that constrains answer generation.

4.3.4.1 Context Formatting

Each retrieved chunk is converted into a structured text block containing identifiers and

timestamps. This preserves temporal grounding and allows the generated answer to be traced back to the source transcript.

```python
def format_flat_chunks(documents: List[Document]) -> str:

    context_parts = []
    for i, doc in enumerate(documents, 1):
        chunk_id = doc.metadata.get('chunk_id', 'unknown')
        start_time = doc.metadata.get('start_time', '')
        end_time = doc.metadata.get('end_time', '')

        header = f"[Chunk {i} - ID: {chunk_id}"
        if start_time and end_time:
            header += f", Time: {start_time} -> {end_time}"
        header += "]"

        context_parts.append(f"{header}\n{doc.page_content}")

    return "\n\n".join(context_parts)
```

Figure 4.28: Chunk formatting with individual metadata headers

```
[Chunk 1 - ID: 42, Time: 00:05:23 -> 00:05:45]
Speaker A discusses the project timeline and key milestones...

[Chunk 2 - ID: 43, Time: 00:05:45 -> 00:06:08]
Speaker B responds with concerns about resource allocation...
```

Figure 4.29: Output format example

```python
def format_chunks_with_citations(documents) -> Dict[str, any]:

    context = format_retrieved_chunks(documents)

    # Flatten groups to individual chunks for source citations
    if documents and isinstance(documents[0], dict):
        # Grouped format - flatten for citations
        flat_docs = []
        for group in documents:
            flat_docs.extend(group['chunks'])
        documents = flat_docs

    # Generate citations
    sources = []
    for i, doc in enumerate(documents, 1):
        # Create a preview (first 100 characters)
        preview = doc.page_content[:100]
        if len(doc.page_content) > 100:
            preview += "..."

        citation = {
            'chunk_number': i,
            'chunk_id': doc.metadata.get('chunk_id', 'unknown'),
            'preview': preview,
            'full_text': doc.page_content,
            'source': doc.metadata.get('source', 'unknown'),
            'transcript_id': doc.metadata.get('transcript_id', 'unknown'),
            'start_time': doc.metadata.get('start_time', ''),
            'end_time': doc.metadata.get('end_time', '')
        }
        sources.append(citation)

    return {
        'context': context,
        'sources': sources
    }
```

Figure 4.30: Citation formatting for user transparency and source attribution

4.3.4.2 Prompt Templates

The prompt layer uses LangChain's chat-prompt abstractions to combine the system instruction, retrieved context, optional chat history, and the user's question. [29, 53, 54] In the current system, route-aware retrieval can also inject transcript-level summary information and higher-level framing for vague or overview-style questions.

```
RAG_SYSTEM_MESSAGE = """
You are a helpful assistant. You have access to relevant
excerpts from a transcript:

{retrieved_context}

Use these excerpts to answer the user's questions. If the
answer cannot be found in the provided excerpts, say so clearly.
Do not make up information not present in the excerpts.
"""
```

Figure 4.31: Grounded QA system message template defining assistant role and constraints

```
from langchain_core.prompts import ChatPromptTemplate

def get_rag_chat_prompt():
    """Create RAG chat prompt template with chat history support."""
    return ChatPromptTemplate.from_messages([
        ("system", RAG_SYSTEM_MESSAGE),
        ("placeholder", "{chat_history}"),
        ("human", RAG_HUMAN_MESSAGE)
    ])
```

Figure 4.32: Complete RAG chat prompt construction with history support

```
from langchain_core.messages import HumanMessage, AIMessage

# Example chat history structure
chat_history = [
    HumanMessage(content="What is the project timeline?"),
    AIMessage(content="The project timeline spans 6 months..."),
    HumanMessage(content="Who is responsible for Phase 2?"),
    AIMessage(content="Speaker B is responsible for Phase 2...")
]

# Injected into prompt via placeholder
prompt.invoke({
    "retrieved_context": formatted_context,
    "chat_history": chat_history,
    "question": "When does Phase 2 start?"
})
```

Figure 4.33: Chat history structure for multi-turn conversation support

```python
# Streaming invocation for real-time display
for chunk in chain.stream({
    "retrieved_context": formatted_context,
    "chat_history": chat_history,
    "question": user_question
}):
    # Print each token as it arrives
    print(chunk, end="", flush=True)

print()  # Newline after complete response
```

Figure 4.34: Streaming response handling for real-time token display

4.4 Keyword-Based Search Implementation

The keyword-based search subsystem complements the chatbot by prioritising direct retrieval rather than answer generation. It is intended for users who want inspectable search results, such as all mentions of a product, statements by a specific speaker, or snippets within a particular date range.

4.4.1 Keyword-First Retrieval Design

The search bar reuses the same retrieval primitives as the RAG subsystem, namely vector search, BM25, phonetic search, fuzzy matching, RRF fusion, and optional reranking. However, the orchestration is intentionally different. BM25 is weighted more heavily so that exact lexical matches dominate when users search for names, brands, technical terms, or other explicit keywords.

Unlike the chatbot, which returns a flat set of supporting chunks for answer generation, the search bar groups results by transcript. This design makes it easier for users to identify which transcripts are most relevant before examining the individual snippets within each transcript.

4.4.2 Two-Phase Search Flow

The search subsystem performs retrieval in two phases. In Phase 1, the system ranks transcripts using summary-derived documents only. This provides a transcript-level relevance signal that is cleaner than mixing all low-level content chunks together. Summary chunks are especially useful here because they represent the high-level contents of a transcript and can therefore act as transcript proxies.

In Phase 2, the selected transcripts are searched again at chunk level. Retrieval is scoped to one transcript at a time, the fused results are reranked, and the final top chunks are returned as grouped results. Summary documents are excluded from the final displayed hits

because users expect timestamped transcript evidence rather than generated summary text in the result list.

Placeholder: implemented keyword-based search flow.
The figure should show filter parsing, Phase 1 transcript ranking using summary chunks, Phase 2 per-transcript chunk retrieval and reranking, and grouped output by transcript.

Figure 4.35: Placeholder for the implemented keyword-based search flow

4.4.3 Date and Speaker Filtering

The search bar supports natural-language date filters and speaker filters in addition to free-text queries. Date expressions are parsed into time ranges using the dateparser library, then matched against transcript upload metadata to ensure all relevant transcripts are considered before chunk-level retrieval. [45] Speaker filtering relies on speaker metadata extracted during transcript parsing and mapped onto the corresponding chunk metadata, enabling transcript-scoped filtering by one or more speakers.

4.4.4 Relationship to the RAG Subsystem

Although the search bar and chatbot share the same retrieval building blocks, they are optimised for different interaction styles. The RAG subsystem uses retrieval to assemble grounded context for answer generation, whereas the search subsystem uses retrieval to return directly inspectable snippet lists. This separation avoids overloading the chatbot interface with direct lookup tasks while still allowing both subsystems to benefit from the same shared indices and metadata.

4.5 Stretch Goals and Deployment

This section will be completed in the final report once the deployment configuration and remaining evaluation work have been finalized.

Chapter 5

Evaluation and Results

This chapter presents the evaluation pipeline implementation, experimental results, and discussion used to assess the retriever system and its end-to-end performance.

5.1 Evaluation Pipeline Implementation

A rigorous evaluation framework was developed to assess both the quality and performance of the RAG system. The evaluation pipeline comprises dataset generation, metric computation, and comparative benchmarking components.

This evaluation framework is designed to operate automatically, eliminating the need for manual human intervention. Traditional evaluation approaches require extensive manual effort, including human annotation of ground truth question-answer pairs, manual inspection of retrieved document relevance, and subjective assessment of generated answer quality [55]. Such manual processes are time-consuming, costly, and difficult to scale across large transcript corpora.

With the advancement of LLM, it is now feasible to design an automated evaluation framework that leverages LLM capabilities for both dataset generation and answer quality assessment [56]. The proposed pipeline automates ground truth generation using an LLM to synthesize contextually relevant question-answer pairs from transcript excerpts (Section 5.1.1), computes retrieval metrics algorithmically through temporal overlap detection (Section 5.1.2), and employs an LLM judge to evaluate semantic correctness of generated answers (Section 5.1.3). This automation enables rapid iteration during system development, facilitates reproducible evaluation across configuration changes, and scales efficiently to accommodate additional transcripts without proportional increases in human labor.

5.1.1 Dataset Generation Methodology

Ground truth evaluation datasets were generated automatically using an LLM-based approach. The generation process follows a structured pipeline that transforms raw SRT transcripts into question-answer pairs with temporal metadata for retrieval evaluation.

5.1.1.1 Context Window Grouping

The grouping algorithm aggregates small SRT chunks into larger context windows suitable for LLM processing. The grouping process employs a greedy first-fit algorithm that iterates through SRT chunks sequentially, accumulating text until a maximum character

threshold is reached.

The algorithm maintains temporal boundaries by tracking the start and end timestamps of each grouped window, ensuring that generated questions can be linked back to specific time ranges in the source transcript. This temporal metadata is critical for subsequent retrieval evaluation.

Figure 5.1 illustrates the transformation pipeline from raw SRT transcripts to a grouped window:

```
Chunk 1:
Timestamp: 00:01:30,000 --> 00:01:45,000
Text: "The project timeline spans six months,
       from January to June."

Chunk 2:
Timestamp: 00:01:45,500 --> 00:02:00,200
Text: "We need to complete Phase 1 by the end
       of February."

Chunk 3:
Timestamp: 00:02:00,800 --> 00:02:15,300
Text: "Speaker B is responsible for coordinating
       the deliverables."

           ↓ (Grouping Algorithm)

start_time: 00:01:30,000
end_time: 00:02:15,300
text: "The project timeline spans six months, from
       January to June. We need to complete Phase 1
       by the end of February. Speaker B is
       responsible for coordinating the deliverables."
chunk_indices: [0, 1, 2]

           ↓ (LLM QA Generation)
```

Figure 5.1: Example transformation from raw SRT transcript chunks to a grouped window with temporal metadata

5.1.1.2 LLM API Call for QA Generation

The QA generation leverages DeepSeek's chat API through LangChain's OpenAI-compatible interface. Figure 5.2 shows the chain initialization:

```python
def initialize_generator_chain():

    llm = ChatOpenAI(
        model="deepseek-chat",
        openai_api_base=DEEPSEEK_BASE_URL,
        openai_api_key=DEEPSEEK_API_KEY,
        temperature=0.7
    )

    prompt = get_qa_generation_prompt()

    return prompt | llm | StrOutputParser()
```

Figure 5.2: LangChain pipe syntax for QA generation chain

The LangChain pipe operator chains three components: the prompt template, the LLM, and the string output parser. The temperature of 0.7 was selected to balance creativity in question formulation with factual grounding. Lower temperatures (e.g., 0.2) produced repetitive questions, while higher temperatures (e.g., 1.0) occasionally generated questions not directly answerable from the context.

5.1.1.3 Generation Prompt Template

The generation prompt template specifies requirements for question-answer pairs. Figure 5.3 shows the prompt template:

```
f"""Based on the following transcript excerpt, generate 2-3 question-answer pairs.

Transcript:
{context}

Requirements:
1. Questions should be specific and answerable from the text
2. Include a mix of detail-oriented and conceptual questions
3. Answers should be concise but complete
4. Questions should be self-contained (no unresolved pronouns)

Output format (JSON without markdown):
[
  {{
    "question": "...",
    "answer": "..."
  }},
  ...
]
"""
```

Figure 5.3: QA generation prompt template with requirements and format

The prompt template enforces three critical requirements. First, questions must be self-contained, avoiding pronouns like "he," "she," or "it" without clear antecedents. This ensures questions can be answered without additional context beyond the retrieved chunks.

Second, answers must be directly supported by the provided text, preventing hallucination. Third, the output must be valid JSON, enabling automated parsing and validation.

The template requests 2-3 question-answer pairs per context window, balancing dataset size with question quality. Requesting more pairs per window (e.g., 5-7) resulted in lower-quality questions that repeated similar information.

5.1.1.4 JSON Output Format and Metadata Enrichment

Each generated QA pair is enriched with three metadata fields. The source_start and source_end fields capture the temporal boundaries of the context window from which the question was generated. This temporal metadata enables retrieval relevance checking during evaluation by allowing the system to verify whether retrieved chunks overlap with the ground truth time range. The context field stores the full context window text, enabling manual validation of question quality and answer correctness.

Figure 5.4 shows an example of the ground truth JSON format:

```
[
  {
    "question": "What is the title of the passage?",
    "answer": "The title is 'The North Wind and the Sun'.",
    "source_start": "00:00:00,640",
    "source_end": "00:03:19,180",
    "context": "full text of source context window..."
  }
]
```

Figure 5.4: Ground truth JSON format example with timestamps

Table 5.1 shows the ground truth schema field descriptions:

Field	Type	Description
question	string	Self-contained question derived from context
answer	string	Ground truth answer from source text
source_start	string	Start timestamp in SRT format (HH:MM:SS,mmm)
source_end	string	End timestamp in SRT format
context	string	Full context window text used for generation

Table 5.1: Ground truth schema field descriptions

5.1.2 Retrieval Metrics

Retrieval metrics evaluate the system's ability to find relevant chunks from the transcript before answer generation. Two complementary metrics are used: Hit Rate (recall-oriented) and Mean Reciprocal Rank (ranking-oriented).

5.1.2.1 Hit Rate Computation

Definition: Proportion of queries where at least one retrieved chunk has temporal overlap with the ground truth timestamp range [27]. The formula is shown in Figure 5.5:

```
Hit Rate = (Number of Hits) / (Total Queries)

where Hit = any retrieved chunk temporally overlaps with ground truth
```

Figure 5.5: Hit Rate formula for retrieval evaluation

Hit Rate is computed through temporal overlap detection between retrieved chunks and ground truth time ranges. Figure 5.6 shows the temporal overlap detection algorithm:

```python
def check_retrieval_relevance(
    retrieved_chunk_sources: List[Dict],
    ground_truth_start: str,
    ground_truth_end: str
) -> Tuple[bool, int]:

    gt_start = parse_timestamp(ground_truth_start)
    gt_end = parse_timestamp(ground_truth_end)

    for i, source in enumerate(retrieved_chunk_sources, 1):
        chunk_start = parse_timestamp(source['start_time'])
        chunk_end = parse_timestamp(source['end_time'])

        # Check for overlap
        # Overlap happens if chunk_start < gt_end AND chunk_end > gt_start
        if chunk_start < gt_end and chunk_end > gt_start:
            return True, i

    return False, 0
```

Figure 5.6: Interval intersection algorithm for retrieval relevance checking

The algorithm implements standard interval intersection logic: two intervals [a,b] and [c,d] overlap if and only if a $<$ d AND b $>$ c. For example, if a QA pair was generated from context spanning [10s, 20s] and a retrieved chunk covers [15s, 25s], the intervals overlap and the chunk is considered relevant. The function returns both a boolean hit flag and the 1-indexed rank of the first relevant chunk, enabling simultaneous Hit Rate and MRR computation.

5.1.2.2 Mean Reciprocal Rank (MRR)

Definition: Average of reciprocal ranks of the first relevant result, as illustrated in Figure 5.7 [9]:

```
MRR = (1 / |Q|) * Σ (1 / rank_i)

where:
  Q = set of queries
  rank_i = position of first relevant chunk for query i (1-indexed)
  rank_i = 0 if no relevant chunk found (contributes 0 to sum)
```

Figure 5.7: Mean Reciprocal Rank (MRR) formula and definition

If a hit is detected, the MRR contribution is computed as 1.0 divided by the rank. An MRR of 1.0 indicates perfect retrieval performance, where the first relevant result is consistently ranked at position 1. An MRR of 0.5 implies that the first relevant result appears at rank 2 on average, while an MRR of 0.333 suggests it appears at rank 3 on average. Conversely, an MRR of 0.0 indicates retrieval failure, where no relevant results are returned for the evaluated queries.

MRR provides a more nuanced assessment than Hit Rate by penalizing lower-ranked relevant results. A system with Hit Rate = 1.0 and MRR = 0.5 successfully retrieves relevant chunks for all queries but often places them at rank 2 or lower, indicating suboptimal ranking quality.

5.1.3 Generation Metrics

Generation metrics evaluate the quality of answers produced by the LLM after retrieval. Two complementary metrics are used: F1 Score (lexical overlap) and LLM Judge Accuracy (semantic correctness).

5.1.3.1 F1 Score Calculation

Definition: Harmonic mean of precision and recall based on token-level overlap between generated and ground truth answers, as shown in Figure 5.8 [27]:

```
Precision = (Common Tokens) / (Predicted Tokens)
Recall = (Common Tokens) / (Ground Truth Tokens)
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

Figure 5.8: F1 score calculation with precision and recall

F1 Score computation requires token-level comparison between generated and ground truth answers. Figure 5.9 shows the normalization process:

```python
def calculate_f1_score(prediction: str, ground_truth: str) -> float:
    """Calculates F1 score based on token overlap."""
    def normalize_text(text):
        return re.sub(r'\W+', ' ', text.lower()).strip()

    pred_tokens = normalize_text(prediction).split()
    gt_tokens = normalize_text(ground_truth).split()

    if not pred_tokens or not gt_tokens:
        return 0.0
```

Figure 5.9: Text preprocessing for token-level comparison

To enable robust token-level comparison, a lightweight normalization step is applied to both texts prior to metric computation. This process converts all characters to lowercase, removes punctuation and other non-word symbols via regex, and collapses repeated whitespace. This focuses the evaluation on lexical content rather than formatting differences.

Figure 5.10 shows the F1 computation using Counter intersection:

```python
    common_tokens = collections.Counter(pred_tokens) & collections.Counter(gt_tokens)
    num_same = sum(common_tokens.values())

    if num_same == 0:
        return 0.0

    precision = num_same / len(pred_tokens)
    recall = num_same / len(gt_tokens)

    return 2 * (precision * recall) / (precision + recall)
```

Figure 5.10: Precision, recall, and harmonic mean calculation with Counter intersection

The Counter intersection operation returns the minimum count for each token present in both prediction and ground truth. For example, if the prediction contains ["apple", "apple", "banana"] and the ground truth contains ["apple", "banana", "banana"], the intersection yields {"apple": 1, "banana": 1}. The num_same variable sums these counts to get the total number of overlapping tokens.

Precision measures what fraction of the predicted tokens are correct. Recall measures what fraction of the ground truth tokens were predicted. F1 is the harmonic mean of precision and recall, providing a balanced measure that penalizes both over-generation (low precision) and under-generation (low recall).

The F1 score provides a balanced measure of lexical overlap between prediction and ground truth, but may underestimate semantic correctness when valid paraphrasing or synonym substitution is present. For example, "quarreled" vs "argued" would reduce F1 despite semantic equivalence.

5.1.3.2 LLM Judge Accuracy (End-to-End Accuracy)

Definition: Binary semantic correctness evaluated by an LLM judge, assessing whether the generated answer is semantically consistent with the ground truth [57]. Figure 5.11 shows the prompt template:

```
f"""Question: {question}
Ground Truth: {ground_truth}
Generated Answer: {prediction}

Is the Generated Answer semantically correct and consistent with the Ground Truth?
Respond with ONLY one word: "CORRECT" or "INCORRECT".

Response:"""
```

Figure 5.11: LLM judge prompt template for semantic correctness evaluation

The LLM judge receives a formatted prompt containing the question, ground truth answer, and generated prediction. The prompt explicitly instructs the judge to output only "CORRECT" or "INCORRECT" based on semantic consistency. The response is parsed using a hierarchical decision logic: if "CORRECT" appears without "INCORRECT", the answer is marked correct; if "INCORRECT" appears, it is marked incorrect; otherwise, a fallback heuristic checks for "CORRECT" in the content. This handles edge cases where the judge provides additional explanation alongside the verdict.

The judge LLM uses temperature 0.0 for deterministic evaluation, reducing variability in scoring. Error handling returns False on judge failure, taking a conservative approach that prefers false negatives to false positives.

Both F1 and LLM Judge metrics are computed for each generated answer and accumulated for averaging. This dual evaluation provides complementary perspectives: F1 captures lexical fidelity while LLM Judge captures semantic correctness.

5.1.4 Separation of Retrieval and Generation Evaluation

The evaluation framework deliberately separates retrieval and generation metrics to enable diagnostic analysis:

Computed Before Generation: Hit Rate and MRR. These metrics do not depend on the generated answer and evaluate only the retrieval pipeline's ability to find relevant chunks.

Computed After Generation: F1 Score and LLM Judge. These metrics evaluate the LLM's ability to extract and synthesize information from the retrieved context, holding the retrieved chunks constant.

This separation enables precise failure attribution. Figure 5.12 illustrates Scenario 1 (retrieval failure):

Scenario 1: Retrieval Failure

```
{
  "retrieval_hit": false,
  "retrieval_rank": 0,
  "is_correct": false,
  "f1_score": 0.0
}
```

Figure 5.12: Retrieval failure scenario diagnostic example

Retrieval pipeline failed to find relevant chunks. One example action is to increase the retriever's top-k value and re-evaluate performance, in combination with adjusting fusion weights and/or refining the reranker configuration.

Figure 5.13 shows Scenario 2 (generation failure):

Scenario 2: Generation Failure

```
{
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.42
}
```

Figure 5.13: Generation failure scenario with successful retrieval

Retrieval succeeded, but the LLM failed to extract the correct answer from the retrieved context. One example action is to refine the prompt (e.g., by adding explicit extraction instructions or structured output formats); additionally, we can experiment with alternative LLMs and temperature settings to improve answer reliability.

Figure 5.14 illustrates Scenario 3 (perfect performance):

Scenario 3: Perfect Performance

```
{
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": true,
  "f1_score": 0.87
}
```

Figure 5.14: Perfect performance scenario for both retrieval and generation

Both retrieval and generation working optimally.

5.1.5 Speed Evaluation Pipeline

This section introduces a lightweight speed evaluation pipeline that is separated from the main evaluation pipeline to enable an unbiased latency comparison between the RAG-based approach and a Pure LLM baseline. The pipeline reuses the same ground truth dataset described in Section 5.1.1, including identical questions, reference answers, and temporal metadata, to ensure consistency across evaluations. This separation is necessary due to prompt caching effects in modern LLM services, which can artificially distort latency measurements under sequential execution [58]. The mitigation strategy adopted to address this issue is discussed in detail in Section 5.1.5.1.

5.1.5.1 Interleaved Execution Design

The speed evaluation pipeline addresses prompt caching bias through interleaved execution.

LLM services cache recently seen prompts to improve efficiency [58]. A sequential benchmarking setup could unintentionally allow the Pure LLM baseline to benefit from warmed caches, thereby underestimating its true latency and producing unfair comparisons. To address this, the pipeline alternates between RAG and Pure LLM execution on a per-question basis, ensuring both methods experience similar caching conditions and enabling a more reliable and unbiased measurement of relative runtime performance, as illustrated in Figure 5.15.

```
Query 1: RAG → Pure LLM
Query 2: RAG → Pure LLM
Query 3: RAG → Pure LLM
...
```

Figure 5.15: Interleaved execution pattern for unbiased speed benchmarking

5.1.5.2 Warmup Phase

The evaluation pipeline includes a warmup phase, which is designed to reduce cold-start effects and improve the reliability of latency benchmarking. Specifically, it triggers just-in-time (JIT) compilation and warms up the Python runtime and dependent libraries, initializes and primes the LLM, embedding, and reranker services, and preloads model weights into GPU memory to avoid first-run loading overhead. In addition, it establishes and stabilizes HTTP connections to mitigate transient network latency and connection setup costs during the measured runs.

Warmup results are excluded from timing statistics to measure steady-state performance.

5.1.6 Statistical Computation

5.1.6.1 Basic Statistics

For each approach (RAG and Pure LLM), the following statistics are computed, as shown in Table 5.2:

Metric	Description
Mean	Average query time across all timed queries
Median	Middle value (robust to outliers)
Min	Best-case performance (fastest query)
Max	Worst-case performance (slowest query)
Std Dev	Standard deviation (consistency measure)

Table 5.2: Statistical metrics table for latency analysis

5.1.6.3 Comparative Metrics

Direct comparison between RAG and Pure LLM is quantified through several metrics, as illustrated in Table 5.3:

Metric	Formula	Interpretation
Speedup Factor	Pure LLM Mean / RAG Mean	>1: RAG faster, <1: Pure LLM faster, =1: equivalent
Time Difference (s)	Pure LLM Mean - RAG Mean	Absolute time saved per query
Percentage Faster	(Difference / Pure LLM Mean) × 100	Relative improvement percentage

Table 5.3: Comparative metrics between RAG and Pure LLM approaches

5.2 Experimental Results

This section presents the empirical results from evaluating the RAG system across four diverse transcripts, covering both quality metrics (retrieval and generation performance) and speed performance (latency comparison with Pure LLM baseline).

It should be noted that the results reported in this section are not final, as the experiments were conducted using a locally hosted LLM for development purpose, which is significantly smaller than the self-hosted LLM deployed on the Meadow9+ production server. Consequently, the absolute latency and generation performance observed here may not directly reflect production-level behavior, and further benchmarking on the target deployment environment is required for conclusive comparison.

5.2.1 RAG Quality Metrics

5.2.1.1 Per-Transcript Results

Table 5.4 presents the per-transcript RAG quality metrics:

Transcript	Duration	Questions	Hit Rate	MRR	Accuracy	F1 Score
Transcript 1	00:08:21	9	1.000	1.000	0.889	0.677
Transcript 2	01:59:40	30	0.867	0.867	0.600	0.476
Transcript 3	00:08:14	8	1.000	1.000	1.000	0.613
Transcript 4	00:36:14	15	1.000	1.000	0.667	0.446
Aggregate	**02:52:29**	**62**	**0.967**	**0.967**	**0.789**	**0.553**

Table 5.4: Per-transcript RAG quality metrics aggregated results

5.2.1.2 Aggregate Interpretation

Retrieval Performance:

- **Hit Rate: 96.7%:** 60 out of 62 queries successfully retrieved at least one temporally relevant chunk
- **MRR: 0.967:** The high average reciprocal rank indicates that relevant chunks typically appear at rank 1

The near-perfect retrieval metrics validate the hybrid search architecture. Only Transcript 2 (the longest at almost 2 hours) exhibited any retrieval failures, with 4 out of 30 queries missing relevant chunks.

Generation Performance:

- **Accuracy: 78.9%:** LLM Judge marked 49 out of 62 generated answers as semantically correct
- **F1 Score: 0.553:** Moderate token-level overlap between generated and ground truth answers

The divergence between Accuracy and F1 (0.789 vs 0.553) indicates that the LLM frequently paraphrases or synthesizes information rather than copying text verbatim from retrieved chunks. This is desirable behavior, as it demonstrates understanding rather than mere extraction.

5.2.1.4 Detailed Example: Success Case

Figure 5.16 shows a detailed success case:

```
{
  "question": "According to the passage, what did the North Wind and the
               Sun quarrel about?",
  "ground_truth": "The North Wind and the Sun quarreled about which of
                   them was the stronger.",
  "generated_answer": "According to the passage, the North Wind and the
                       Sun quarrelled about which of them was the
                       stronger.",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": true,
  "f1_score": 0.8125,
  "sources": [
    {
      "chunk_id": 1,
      "start_time": "00:00:15,140",
      "end_time": "00:01:31,240",
      "rerank_score": 3.172,
      "preview": "the north wind and the sun had a quarrel about which of
                  them was the stronger..."
    }
  ]
}
```


Figure 5.16: Detailed success case example with perfect retrieval and generation

Analysis:

- **Perfect retrieval:** Relevant chunk ranked at position 1 with high rerank score (3.172)
- **Near-perfect generation:** Minor spelling variation ("quarreled" vs "quarrelled") does not affect semantic correctness
- **High F1 (0.8125):** Strong lexical overlap due to minimal paraphrasing
- **LLM Judge:** Correct: Semantic equivalence recognized despite spelling difference

This represents optimal system performance across all stages.

5.2.1.5 Detailed Example: Failure Case

Figure 5.17 shows a failure case:

```
{
  "question": "What does the speaker find unusual about the blind box packaging?",
  "ground_truth": "The speaker notes that the box does not say 'Laboo' or
                   'Labu' anywhere, but instead says 'exciting macaron' and
                   'the monsters', and that the top sticker feels tampered
                   with.",
  "generated_answer": "Based on the provided excerpts, the speaker finds that
                       the packaging of the blind box \"smells kind of
                       artificial\" and specifically \"like a carpet\"
                       (Chunk 2).",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.291
}
```


Figure 5.17: Failure case example with retrieval success but generation error

Analysis:

- **Perfect retrieval:** Relevant chunk ranked at position 1
- **Generation failure:** Model focused on smell observations instead of labeling/branding issues
- **Low F1 (0.291):** Minimal lexical overlap - completely different aspects mentioned
- **LLM Judge:** Incorrect: Answer addresses wrong aspect of "unusual"

This is a true generation failure where retrieval succeeded in finding relevant context, but the LLM selected less salient information (smell) over the main point (missing brand name). The model may have latched onto sensory details in the retrieved chunk rather than the more significant observation about brand labeling. Such errors may be mitigated through prompt engineering that explicitly emphasizes answering the core question over incidental attributes, context window optimization to reduce distracting noise, fine-tuning on examples that require distinguishing primary from secondary information, and lightweight post-processing checks to validate that the generated answer aligns with the intended focus of the query.

5.2.1.6 Detailed Example: False Negative

Figure 5.18 illustrates a false negative example:

```
{
  "question": "What was the hardest decision the speaker had to make?",
  "ground_truth": "The hardest decision was choosing between two job offers:
                   one with NUHS in the memory clinic doing research, and the
                   other at the current place, which they ultimately chose.",
  "generated_answer": "The hardest decision the speaker had to make was
                       choosing between two job offers. One was with a memory
                       clinic doing research, and the other was joining HDB.
                       The speaker had to make this decision within a few days,
                       and in the end, they chose to join HDB, which they are
                       happy with.",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.585
}
```


Figure 5.18: False negative example with more specific generated answer

Analysis:

- **Perfect retrieval:** Relevant chunk ranked at position 1
- **More specific generation:** Generated answer mentions "HDB" explicitly, while ground truth uses vague "current place"
- **Moderate F1 (0.585):** Partial lexical overlap captures the correct structure with added specificity
- **LLM Judge:** Incorrect: Marked as incorrect due to specificity mismatch

This represents a false negative where the generated answer is actually more accurate and informative than the ground truth. The transcript does mention HDB (Housing & Development Board), making this not a hallucination but rather additional specificity. The LLM Judge may have been overly strict in requiring exact phrasing match. This example highlights a common evaluation challenge when model outputs include correct extra details, where vague ground truth annotations (e.g., “current place”) can inadvertently penalize semantically equivalent but more specific answers. To address this, evaluation should incorporate metrics and judging criteria that tolerate acceptable specificity variation, and the ground truth and/or judge prompt can be revised to recognize paraphrases and grounded elaborations as correct.

From observation, such cases are relatively frequent in practice, where correct generations are incorrectly marked as wrong, which may lead to an underestimation of the true accuracy of the RAG pipeline. This issue is planned to be addressed within the current semester through refinements to the evaluation methodology, and the updated approach will be documented in the final report.

5.2.2 Speed Performance

5.2.2.1 Per-Transcript Speed Comparison

Table 5.5 presents the per-transcript speed comparison:

Transcript	Duration	RAG Mean (s)	Pure LLM Mean (s)	Speedup Factor
Transcript 1	00:08:21	7.01	10.03	1.43x
Transcript 2	01:59:40	7.39	51.62	6.99x
Transcript 3	00:08:14	6.70	11.57	1.73x
Transcript 4	00:36:14	7.14	49.63	6.95x
Aggregate	**02:52:29**	**7.15**	**38.53**	**5.39x**

Table 5.5: Per-transcript speed comparison showing speedup factors

5.2.2.2 Speedup vs Transcript Size Analysis

RAG Performance (Stable):

- Mean RAG time ranges from 6.70s to 7.39s across all transcripts
- Retrieval extracts a small, fixed-size context regardless of transcript length
- Generation time depends only on retrieved context size, not source transcript size
- Result: Consistent latency independent of transcript length

Pure LLM Performance (Scales with Size):

- Generation time scales approximately linearly with context size
- Result: Latency grows with transcript length

RAG provides greater speed advantages for longer transcripts, which are common in real-world meeting recording scenarios.

5.2.2.5 Quality vs Speed Trade-Off Analysis

Table 5.6 summarizes the quality-speed trade-off analysis:

Transcript	Speedup	RAG Accuracy	Pure LLM Accuracy	Accuracy Δ	Trade-Off
Transcript 1	1.43x	0.889	0.875	+0.014	Speed + Quality
Transcript 2	6.99x	0.600	0.667	-0.067	Speed » Quality
Transcript 3	1.73x	1.000	1.000	0.000	Speed + Quality
Transcript 4	6.95x	0.667	0.733	-0.066	Speed » Quality

Table 5.6: Quality-speed trade-off analysis across transcript sizes

RAG provides substantial speed improvements (1.4x to 7x) with either comparable or slightly reduced quality. For small transcripts, RAG is strictly superior (faster and equal/better quality). For large transcripts, RAG offers a favorable trade-off with massive speedup for small accuracy reduction.

Chapter 6

Conclusion and Future Work

This chapter will be completed in the final report, once everything has been finalized.

6.1 Future Work

Based on consultations with project supervisors, several areas for future development have been identified.

6.1.1 Limited Coverage of Question Types in the Evaluation Pipeline

The current evaluation pipeline is designed under the assumption that users ask highly specific questions that require extracting precise, localized information from a transcript. This is reflected in the construction of the ground truth dataset, where questions are intentionally scoped to narrow spans of evidence. Examples include:

- "What is the example given of a summarized headline about Baldur's Gate 3 players?"
- "What is the rarity of the secret character in the blind box compared to the other characters?"
- "What does the speaker find unusual about the blind box packaging?"

However, the evaluation pipeline does not yet cover broader and more open-ended queries such as "What is being discussed in this meeting?", where the answer may require integrating context across multiple topics and long time ranges [5]. Additional examples of such vague queries include:

- "Can you summarize the main points from this meeting?"
- "What are the key decisions made in this conversation?"
- "What problems or concerns were raised by the team?"
- "What action items or next steps were mentioned?"
- "What is the overall tone or sentiment of the discussion?"

These question types may perform worse under the current retrieval design, as they often require global transcript-level understanding rather than a small set of relevant chunks [5].

Extending the evaluation pipeline to include these high-level summarization and meeting-understanding queries will be addressed in the current semester and incorporated into the final report.

6.1.2 RAG Pipeline Optimization for Ambiguous Queries

The existing RAG pipeline requires enhancement to better handle vague or underspecified questions as described in Section 6.1.1. Proposed improvements include:

- Integration of document summaries into the LLM context to provide high-level contextual information
- Incorporation of neighboring retrieved chunks to maintain semantic coherence across chunk boundaries
- Leveraging the expanded context window available in the Meadow9+ server deployment, which utilizes a larger LLM compared to the locally hosted variant used for development

These enhancements necessitate comprehensive re-evaluation of system latency to ensure performance requirements remain satisfied despite the increased context size [6].

6.1.3 Production Deployment of Retrieval Server

Subject to time constraints, investigation into full production deployment of the retrieval server infrastructure will be undertaken. This includes:

- Infrastructure provisioning and scalability considerations
- Deployment architecture design
- Production monitoring and maintenance protocols

Bibliography

[1] M. Shao, A. Basit, R. Karri, and M. Shafique, "Survey of different large language model architectures: Trends, benchmarks, and challenges," *IEEE Access*, vol. 12, pp. 188 664–188 706, 2024.

[2] Hostinger. (2025) Llm statistics 2025: Adoption, trends, and market insights. [Online]. Available: https://www.hostinger.com/tutorials/llm-statistics

[3] S. K. Dam, N. Sharma, M. Sahay, and M. Shah, "A complete survey on llm-based ai chatbots," *arXiv preprint arXiv:2406.16937*, 2024.

[4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[6] NVIDIA. (2023) Mastering llm techniques: Inference optimization. NVIDIA Technical Blog. [Online]. Available: https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/

[7] K. Kuhn, V. Kersken, B. Reuter, N. Egger, and G. Zimmermann, "Measuring the accuracy of automatic speech recognition solutions," *ACM Transactions on Accessible Computing*, vol. 16, no. 4, pp. 25:1–25:23, 2023.

[8] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2024.

[9] Wikipedia. (2024) Mean reciprocal rank. [Online]. Available: https://en.wikipedia.org/wiki/Mean_reciprocal_rank

[10] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.

[11] R. Nogueira and K. Cho, "Passage re-ranking with bert," *arXiv preprint arXiv:1901.04085*, 2019.

[12] Otter.ai. (2026) Otter transcribe voice notes. [Online]. Available: https://apps.apple.com/na/app/otter-transcribe-voice-notes/id1276437113

[13] ——. (2026) Ai sales notetaker. [Online]. Available: https://otter.ai/sales-agent

[14] ——. (2026) Ai education notetaker for real-time lecture notes. [Online]. Available: https://otter.ai/education-agent

[15] ——. (2026) Otter features. [Online]. Available: https://otter.ai/features

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] Otter.ai Help Center. (2026) Otter AI chat FAQs. [Online]. Available: https://help.otter.ai/hc/en-us/articles/17016733191703-Otter-AI-Chat-FAQs

[18] Anthropic. (2026) Claude code tutorials. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/tutorials

[19] ——. (2026) Claude code settings. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/settings

[20] ——. (2026) Claude code team. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/team

[21] ——. (2026) Manage claude code memory. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/memory

[22] ——. (2026) Using extended thinking. [Online]. Available: https://support.anthropic.com/en/articles/10574485-using-extended-thinking

[23] Meadow9. (2026) Meadow9. [Online]. Available: https://apps.apple.com/au/app/meadow9/id1474941556

[24] ABAX.AI. (2026) Abax ai | custom speech to text. [Online]. Available: https://www.abax.ai/

[25] ABAX.AI PRIVATE LIMITED. (2026) Abax.ai private limited (202128671g). [Online]. Available: https://www.companies.sg/business/202128671G/ABAX-AI-PRIVATE-LIMITED

[26] Meadow9+. (2026) Meadow9+. [Online]. Available: https://apps.apple.com/us/app/meadow9/id6740014005

[27] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979. [Online]. Available: https://search.worldcat.org/title/Information-retrieval/oclc/4494554

[28] LangChain. (2026) Document. [Online]. Available: https://api.python.langchain.com/en/latest/core/documents/langchain_core.documents.base.Document.html

[29] ——. (2026) Prompt templates. [Online]. Available: https://python.langchain.com/docs/concepts/prompt_templates/

[30] ——. (2026) Output parsers. [Online]. Available: https://api.python.langchain.com/en/latest/core/output_parsers.html

[31] FastAPI. (2026) Fastapi. [Online]. Available: https://fastapi.tiangolo.com/

[32] Pydantic. (2026) Pydantic. [Online]. Available: https://docs.pydantic.dev/

[33] Jina AI. (2026) jina-embeddings-v3. [Online]. Available: https://huggingface.co/jinaai/jina-embeddings-v3

[34] BAAI. (2026) bge-reranker-v2-m3. [Online]. Available: https://huggingface.co/BAAI/bge-reranker-v2-m3

[35] Qwen Team. (2026) Qwen3-14b-base. [Online]. Available: https://huggingface.co/Qwen/Qwen3-14B-Base

[36] DeepSeek. (2026) Deepseek api documentation. [Online]. Available: https://api-docs.deepseek.com/

[37] ggml-org. (2026) llama.cpp. [Online]. Available: https://github.com/ggml-org/llama.cpp

[38] LangChain. (2026) Openai embeddings. [Online]. Available: https://python.langchain.com/docs/integrations/text_embedding/openai/

[39] ——. (2026) Chroma vector store. [Online]. Available: https://python.langchain.com/api_reference/chroma/vectorstores/langchain_chroma.vectorstores.Chroma.html

[40] Chroma. (2026) Performance. [Online]. Available: https://docs.trychroma.com/guides/deploy/performance

[41] rank-bm25. (2026) rank-bm25. [Online]. Available: https://pypi.org/project/rank-bm25/

[42] metaphone. (2026) metaphone. [Online]. Available: https://pypi.org/project/metaphone/

[43] RapidFuzz. (2026) Rapidfuzz usage: fuzz. [Online]. Available: https://rapidfuzz.github.io/RapidFuzz/Usage/fuzz.html

[44] wordfreq. (2026) wordfreq. [Online]. Available: https://pypi.org/project/wordfreq/

[45] dateparser. (2026) dateparser documentation. [Online]. Available: https://dateparser.readthedocs.io/en/stable/index.html

[46] pandas. (2026) pandas documentation. [Online]. Available: https://pandas.pydata.org/docs/dev/index.html

[47] NumPy. (2026) Numpy documentation. [Online]. Available: https://numpy.org/doc/2.0/

[48] Matplotlib. (2026) Matplotlib documentation. [Online]. Available: https://matplotlib.org/stable/

[49] pytest. (2026) pytest documentation. [Online]. Available: https://docs.pytest.org/en/9.0.x/

[50] Requests. (2026) Requests: Http for humans. [Online]. Available: https://docs.python-requests.org/en/latest/

[51] P. E. Black. (2021) double metaphone. National Institute of Standards and Technology. [Online]. Available: https://www.nist.gov/dads/HTML/doubleMetaphone.html

[52] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Dokl. Akad. Nauk SSSR*, vol. 163, no. 4, pp. 845–848, 1965. [Online]. Available: https://www.mathnet.ru/eng/dan31411

[53] LangChain. (2026) Chatprompttemplate. [Online]. Available: https://python.langchain.com/api_reference/core/prompts/langchain_core.prompts.chat.ChatPromptTemplate.html

[54] ——. (2026) Messagesplaceholder. [Online]. Available: https://api.python.langchain.com/en/latest/prompts/langchain_core.prompts.chat.MessagesPlaceholder.html

[55] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "Ares: An automated evaluation framework for retrieval-augmented generation systems," *arXiv preprint arXiv:2311.09476*, 2023.

[56] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[57] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-eval: Nlg evaluation using gpt-4 with better human alignment," *arXiv preprint arXiv:2303.16634*, 2023.

[58] OpenAI. (2024) Prompt caching in the api. [Online]. Available: https://openai.com/index/api-prompt-caching/